

What Are We Synchronizing?

ROBERT LEAHY

Core responsibilities aren't code's only responsibilities

Observability is an obvious example.

```
mutable std::mutex m;  
std::size_t packets{0};  
  
void on_packet(...) {  
    // ...  
    const std::lock_guard _(m);  
    ++packets;  
}  
  
std::size_t get_packets() const noexcept {  
    const std::lock_guard _(m);  
    return packets;  
}
```

```
std::atomic<std::size_t> packets;

void on_packet(...) {
    // ...
    packets.fetch_add(1, std::memory_order_relaxed);
}

std::size_t get_packets() const noexcept {
    return packets.load(std::memory_order_relaxed);
}
```

```
std::atomic<std::size_t> packets;

void on_packet(...) {
    // ...
    packets.fetch_add(1, std::memory_order_relaxed);
}

std::size_t get_packets() const noexcept {
    return packets.load(std::memory_order_relaxed);
}
```

- **The unspeakables:** I'll grudgingly and reluctantly talk about the Thing I Said I'd Never Teach That Programmers Should Never Need To Now: relaxed atomics. Don't use them! If you can avoid it. But here's what you need to know, even though it would be nice if you didn't need to know it.



Published by Herb Sutter

Herb Sutter is an author and speaker, and a technical fellow at Citadel Securities. He serves as chair of the Standard C++ Foundation and its conference CppCon, and served as chair of the ISO C++ standards committee from 2002 to 2025. [View all posts by Herb Sutter](#)

Published

2013-02-11

Sometimes the meaning of a value is what it is

Packet count is just a count of packets used for observability, we don't assume it means anything further.

```
template<std::invocable... Fs>
void when_all(Fs&&... fs) {
    std::mutex m;
    std::exception_ptr ex;
    const auto fail = [&]() noexcept {
        const std::lock_guard _(m);
        if (!ex) {
            ex = std::current_exception();
        }
    };

    std::array<std::thread, sizeof...(Fs) - 1> workers;
    // ...
    for (auto&& worker : workers) if (worker.joinable()) worker.join();

    if (ex) {
        std::rethrow_exception(std::move(ex));
    }
}
```

```
template<std::invocable... Fs>
void when_all(Fs&&... fs) {
    std::mutex m;
    std::exception_ptr ex;
    const auto fail = [&]() noexcept {
        const std::lock_guard _(m);
        if (!ex) {
            ex = std::current_exception();
        }
    };

    std::array<std::thread, sizeof...(Fs) - 1> workers;
    // ...
    for (auto&& worker : workers) if (worker.joinable()) worker.join();

    if (ex) {
        std::rethrow_exception(std::move(ex));
    }
}
```

```
template<std::invocable... Fs>
void when_all(Fs&&... fs) {
    std::atomic<bool> failed;
    std::exception_ptr ex;
    const auto fail = [&]() noexcept {

        if (!failed.exchange(true, std::memory_order_relaxed)) {
            ex = std::current_exception();
        }
    };

    std::array<std::thread, sizeof...(Fs) - 1> workers;
    // ...
    for (auto&& worker : workers) if (worker.joinable()) worker.join();

    if (ex) {
        std::rethrow_exception(std::move(ex));
    }
}
```

Modification Order

What is it?

All modifications to a particular atomic object M occur in some particular total order, called the *modification order* of M .

§6.10.2.2 [intro.races]

Vis-à-vis relaxed

An atomic read-modify-write operation always reads the value from the side effect that immediately precedes the side effect of the read-modify-write operation in the modification order.

§32.5.4 [atomics.order]

Effects: Atomically replaces the value pointed to by `this` with desired. Memory is affected according to the value of `order`.
These operations are atomic read-modify-write operations.

§32.5.8.2 [atomic.types.operations]

```
template<std::invocable... Fs>
void when_all(Fs&&... fs) {
    std::atomic<bool> failed;
    std::exception_ptr ex;
    const auto fail = [&]() noexcept {

        if (!failed.exchange(true, std::memory_order_relaxed)) {
            ex = std::current_exception();
        }
    };

    std::array<std::thread, sizeof...(Fs) - 1> workers;
    // ...
    for (auto&& worker : workers) if (worker.joinable()) worker.join();

    if (ex) {
        std::rethrow_exception(std::move(ex));
    }
}
```

Data Race?

Conflicting Evaluations

Two expression evaluations *conflict* if one of them [...] modifies a memory location [...] and the other one [...] reads or modifies the same memory location[.]

§6.10.2.2 [intro.races]

Undefined Behavior

The execution of a program contains a *data race* if it contains two potentially concurrent conflicting actions, at least one of which is not atomic, and neither happens before the other[.] Any such data race results in undefined behavior.

§6.10.2.2 [intro.races]

```
template<std::invocable... Fs>
void when_all(Fs&&... fs) {
    std::atomic<bool> failed;
    std::exception_ptr ex;
    const auto fail = [&]() noexcept {

        if (!failed.exchange(true, std::memory_order_relaxed)) {
            ex = std::current_exception();
        }
    };

    std::array<std::thread, sizeof...(Fs) - 1> workers;
    // ...
    for (auto&& worker : workers) if (worker.joinable()) worker.join();

    if (ex) {
        std::rethrow_exception(std::move(ex));
    }
}
```

Synchronization

Joining a Thread

Synchronization: The completion of the thread represented by `*this` synchronizes with the corresponding successful `join()` return.”

§32.4.3.7 [thread.thread.member]

Happens Before

Certain library calls *synchronize with* other library calls performed by another thread.

[...]

An evaluation *A happens before* an evaluation *B* (or, equivalently, *B happens after A*) if [...] *A* synchronizes with *B* [...]

§6.10.2.2 [intro.races]

```

template<std::invocable... Fs>
void when_all(Fs&&... fs) {
    std::atomic<bool> failed;
    std::exception_ptr ex;
    const auto fail = [&]() noexcept {

        if (!failed.exchange(true, std::memory_order_relaxed)) {
            ex = std::current_exception();
        }
    };

    std::array<std::thread, sizeof...(Fs) - 1> workers;
    // ...
    for (auto&& worker : workers) if (worker.joinable()) worker.join();
    // Every worker thread has ended, side effects of all workers are visible
    if (ex) {
        std::rethrow_exception(std::move(ex));
    }
}

```

Synchronization

Happens Before

An evaluation *A* *happens before* an evaluation *B* (or, equivalently, *B* *happens after* *A*) if either

- *A* is sequenced before *B*, or
- *A* synchronizes with *B*, or
- *A* happens before *X* and *X* happens before *B*.

§6.10.2.2 [intro.races]

Sequenced Before

Every value computation and side effect associated with a full-expression is sequenced before every value computation and side effect associated with the next full-expression to be evaluated.

§6.10.1 [intro.execution]

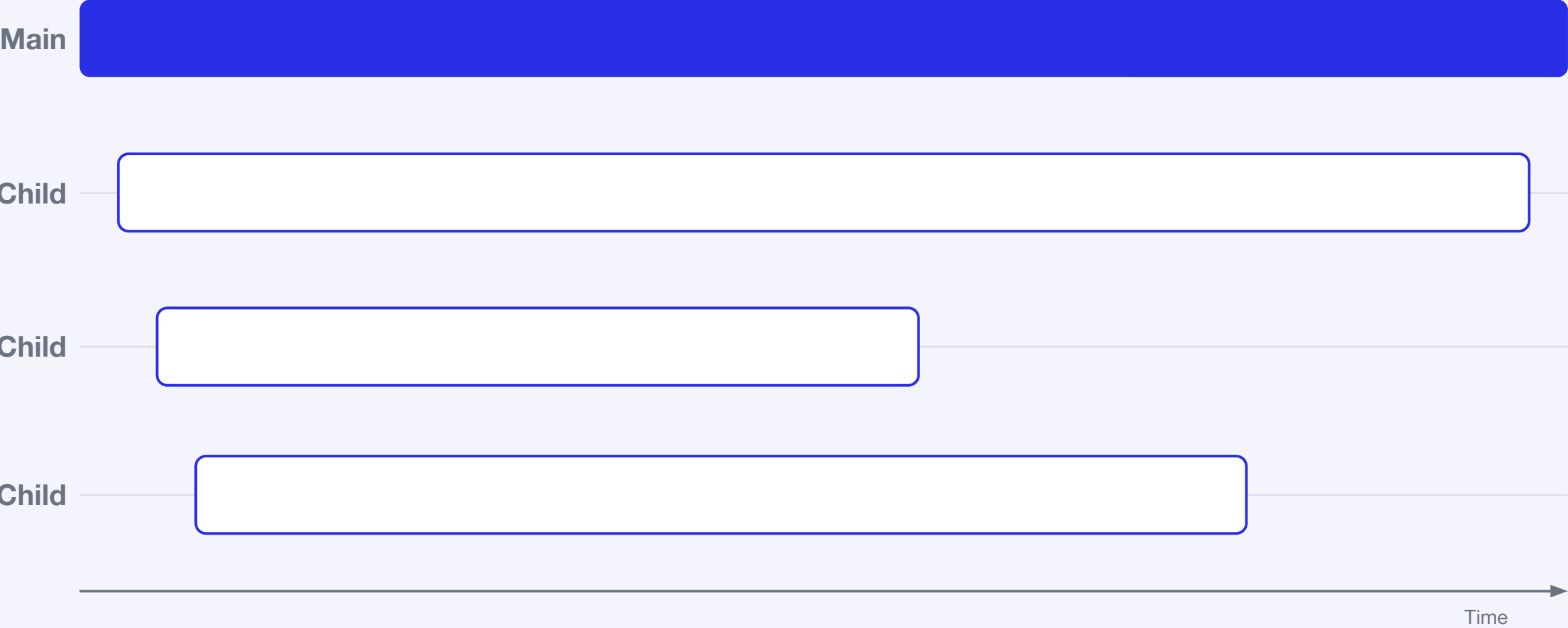
Happens before is a strict partial order

Irreflexive: A does not happen before A

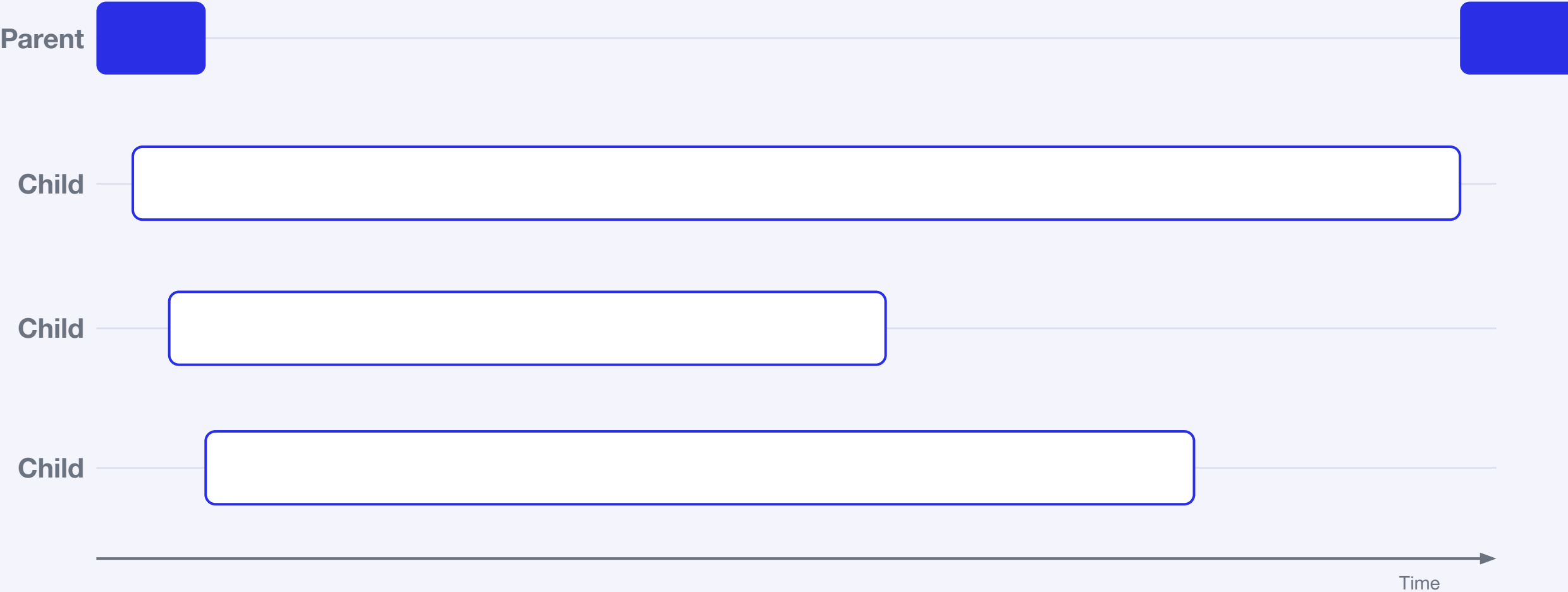
Asymmetric: If A happens before B, B does not happen before A

Transitive: If A happens before B, and B happens before C, A happens before C

Synchronous when_all



Asynchronous when_all



Extending `std::execution`

*Implementing Custom Algorithms
with Senders & Receivers*

Robert Leahy

```
void maybe_complete_() noexcept {  
    // ...  
    const auto children = op.states_->size();  
    const auto arrived =  
        op.arrived_.fetch_add(1, std::memory_order_acq_rel) + 1U;  
    if (arrived != children) {  
        return;  
    }  
    // ...  
}
```

```
void maybe_complete_() noexcept {  
    // ...  
    const auto children = op.states_->size();  
    const auto arrived =  
        op.arrived_.fetch_add(1, std::memory_order_acq_rel) + 1U;  
    if (arrived != children) {  
        return;  
    }  
    // ...  
}
```

Synchronization

`memory_order_acq_rel`

- [...] `memory_order::acq_rel` [...]: a store operation performs a release operation on the affected memory location.
- [...] `memory_order::acq_rel` [...]: a load operation performs an acquire operation on the affected memory location.

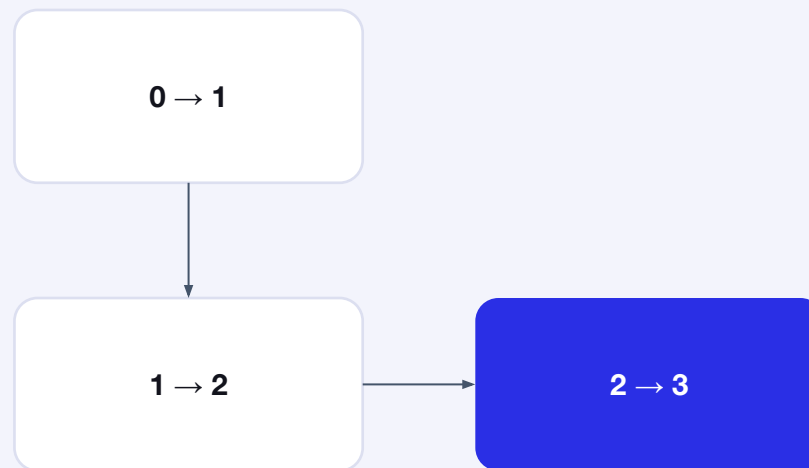
§32.5.4 [atomics.order]

Acquire and Release

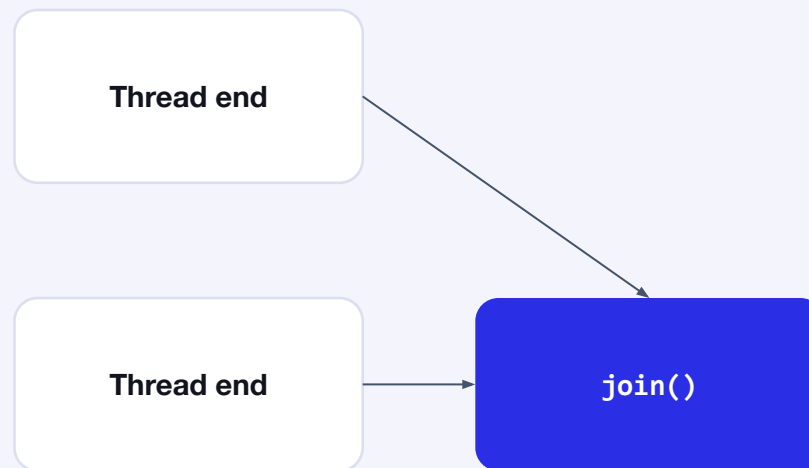
An atomic operation A that performs a release operation on an atomic object M synchronizes with an atomic operation B that performs an acquire operation on M and takes its value from any side effect in the release sequence headed by A .

§32.5.4 [atomics.order]

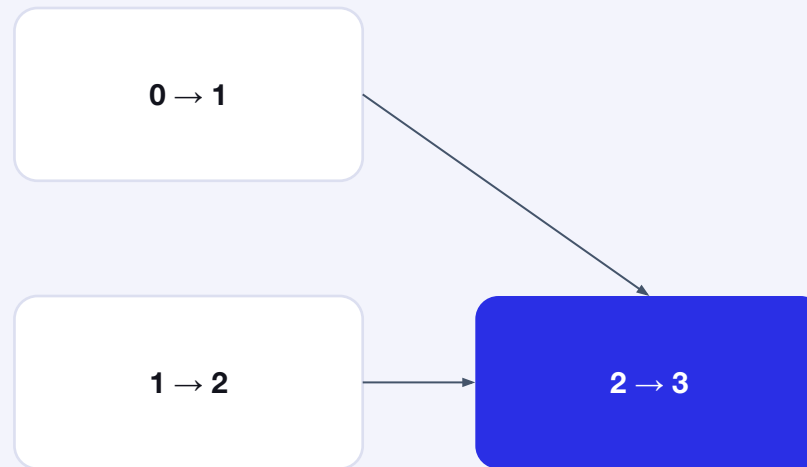
Pairwise Synchronization



Synchronous when_all



Synchronize With Finalization Only



```
void maybe_complete_() noexcept {  
    // ...  
    const auto children = op.states_->size();  
    const auto arrived =  
        op.arrived_.fetch_add(1, std::memory_order_acq_rel) + 1U;  
    if (arrived != children) {  
        return;  
    }  
    // ...  
}
```

```
void maybe_complete_() noexcept {  
    // ...  
    const auto children = op.states_->size();  
    const auto arrived =  
        op.arrived_.fetch_add(1, std::memory_order_release) + 1U;  
    if (arrived != children) {  
        return;  
    }  
    // ...  
}
```

An atomic operation A that performs a **release operation** on an atomic object M synchronizes with an atomic operation B that performs an **acquire operation** on M and takes its value from any side effect in the release sequence headed by A .

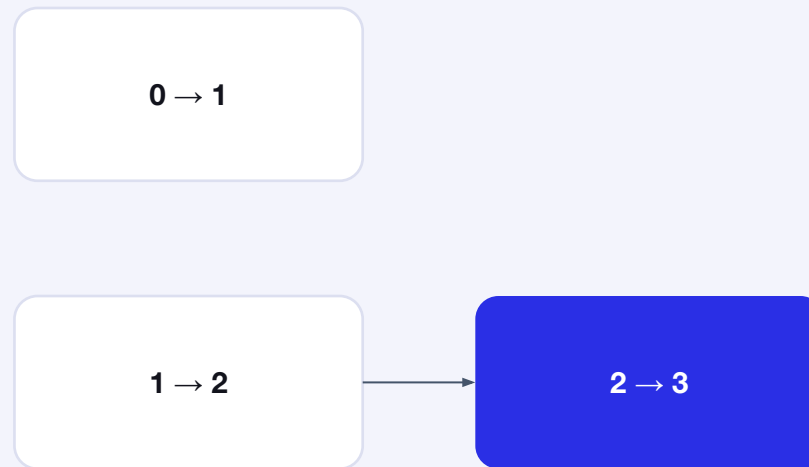
§32.5.4 [atomics.order]

```
void maybe_complete_() noexcept {  
    // ...  
    const auto children = op.states_->size();  
    const auto arrived =  
        op.arrived_.fetch_add(1, std::memory_order_release) + 1U;  
    if (arrived != children) {  
        return;  
    }  
    (void)op.arrived_.load(std::memory_order_acquire);  
    // ...  
}
```

If a side effect X on an atomic object M happens before a value computation B of M , then the evaluation B takes its value from X or from a side effect Y that follows X in the modification order of M .

§6.10.2.2 [intro.races]

Under Synchronized?



Synchronization

Which Values Synchronize?

An atomic operation A that performs a release operation on an atomic object M synchronizes with an atomic operation B that performs an acquire operation on M and **takes its value from any side effect in the release sequence headed by A .**

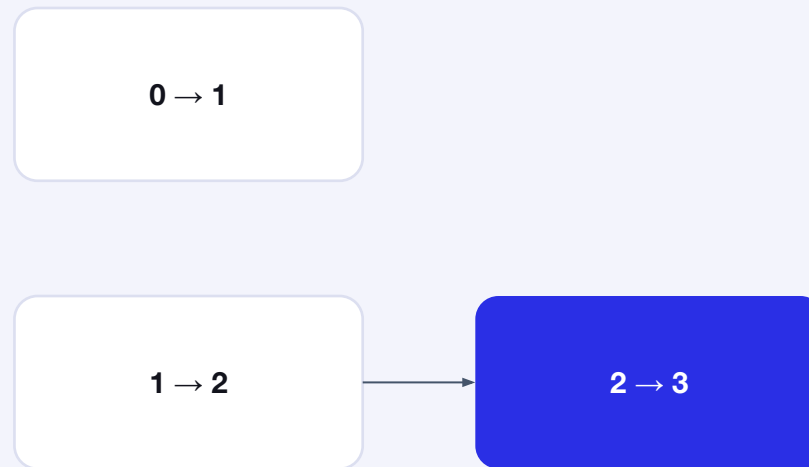
§32.5.4 [atomics.order]

What is a Release Sequence?

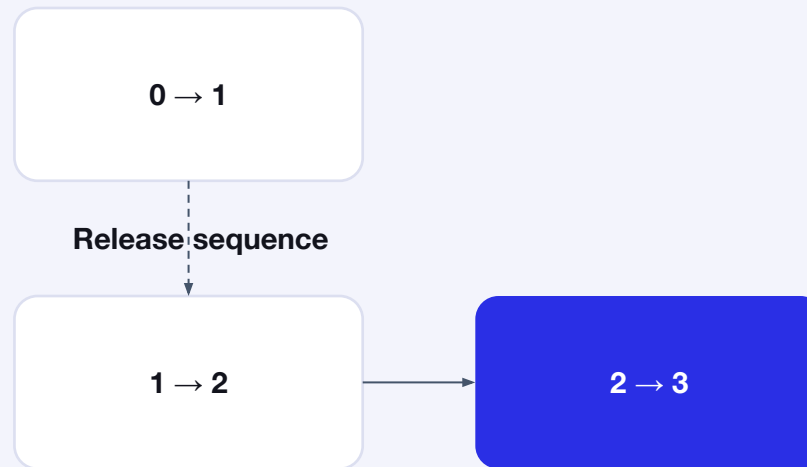
A release sequence headed by a release operation A on an atomic object M is a maximal contiguous sub-sequence of side effects in the modification order of M , where the first operation is A , and every subsequent operation is an atomic read-modify-write operation.

§6.10.2.2 [intro.races]

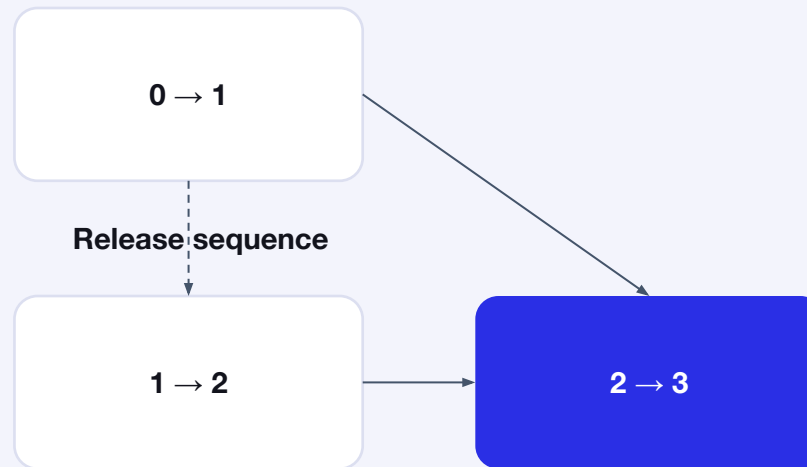
Under Synchronized?



Release Sequence



Happens Before



```
void maybe_complete_() noexcept {  
    // ...  
    const auto children = op.states_->size();  
    const auto arrived =  
        op.arrived_.fetch_add(1, std::memory_order_release) + 1U;  
    if (arrived != children) {  
        return;  
    }  
    (void)op.arrived_.load(std::memory_order_acquire);  
    // ...  
}
```



Stefan @schteppe · 7h



std::remove doesn't remove

std::unique doesn't make it unique

std::async isn't reliably async

std::vector<bool> isn't a vector of bool

std::move doesn't move

std::forward doesn't forward

std::endl isn't just a newline

std::map::operator[] might insert

C++ 💔



45



84



1.3K



41K



boo_ghoul

@rep_movsd



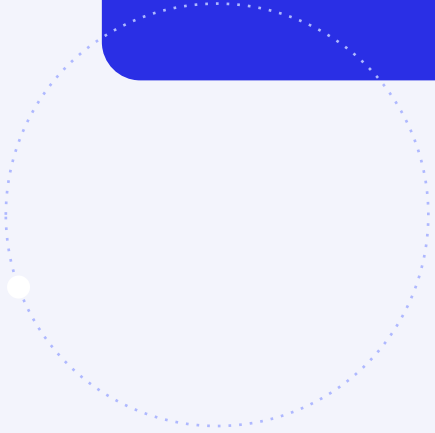
vector::empty doesn't empty it

9:09 AM · Aug 23, 2026 · **3,181** Views

```
void maybe_complete_() noexcept {  
    // ...  
    const auto children = op.states_->size();  
    const auto arrived =  
        op.arrived_.fetch_add(1, std::memory_order_release) + 1U;  
    if (arrived != children) {  
        return;  
    }  
    std::atomic_thread_fence(std::memory_order_acquire);  
    // ...  
}
```

An atomic operation A that is a release operation on an atomic object M synchronizes with an acquire fence B if there exists some atomic operation X on M such that X is sequenced before B and reads the value written by A or a value written by any side effect in the release sequence headed by A .

§32.5.11 [atomics.fences]

A decorative dotted circle with a small white dot at its center, located in the bottom-left corner of the slide.

```
void maybe_complete_() noexcept {  
    // ...  
    const auto children = op.states_->size();  
  
    const auto arrived =  
        op.arrived_.fetch_add(1, std::memory_order_release) + 1U;  
  
    if (arrived != children) {  
        return;  
    }  
  
    std::atomic_thread_fence(std::memory_order_acquire);  
    // ...  
}
```

```
void maybe_complete_() noexcept {  
    // ...  
    const auto children = op.states_->size();  
    // An atomic operation A that is a release operation on atomic object M...  
  
    const auto arrived =  
        op.arrived_.fetch_add(1, std::memory_order_release) + 1U;  
  
    if (arrived != children) {  
        return;  
    }  
  
    std::atomic_thread_fence(std::memory_order_acquire);  
    // ...  
}
```

```
void maybe_complete_() noexcept {  
    // ...  
    const auto children = op.states_->size();  
    // An atomic operation A that is a release operation on atomic object M...  
  
    const auto arrived =  
        op.arrived_.fetch_add(1, std::memory_order_release) + 1U;  
  
    if (arrived != children) {  
        return;  
    }  
    // ...synchronizes with an acquire fence B if there exists some...  
    std::atomic_thread_fence(std::memory_order_acquire);  
    // ...  
}
```

```
void maybe_complete_() noexcept {  
    // ...  
    const auto children = op.states_->size();  
    // An atomic operation A that is a release operation on atomic object M...  
    // ...atomic operation X on M such that X is sequenced before B and...  
    const auto arrived =  
        op.arrived_.fetch_add(1, std::memory_order_release) + 1U;  
  
    if (arrived != children) {  
        return;  
    }  
    // ...synchronizes with an acquire fence B if there exists some...  
    std::atomic_thread_fence(std::memory_order_acquire);  
    // ...  
}
```

```
void maybe_complete_() noexcept {  
    // ...  
    const auto children = op.states_->size();  
    // An atomic operation A that is a release operation on atomic object M...  
    // ...atomic operation X on M such that X is sequenced before B and...  
    const auto arrived =  
        op.arrived_.fetch_add(1, std::memory_order_release) + 1U;  
    // ...reads the value written by A or a value written by any side effect in  
    // the release sequence headed by A.  
    if (arrived != children) {  
        return;  
    }  
    // ...synchronizes with an acquire fence B if there exists some...  
    std::atomic_thread_fence(std::memory_order_acquire);  
    // ...  
}
```

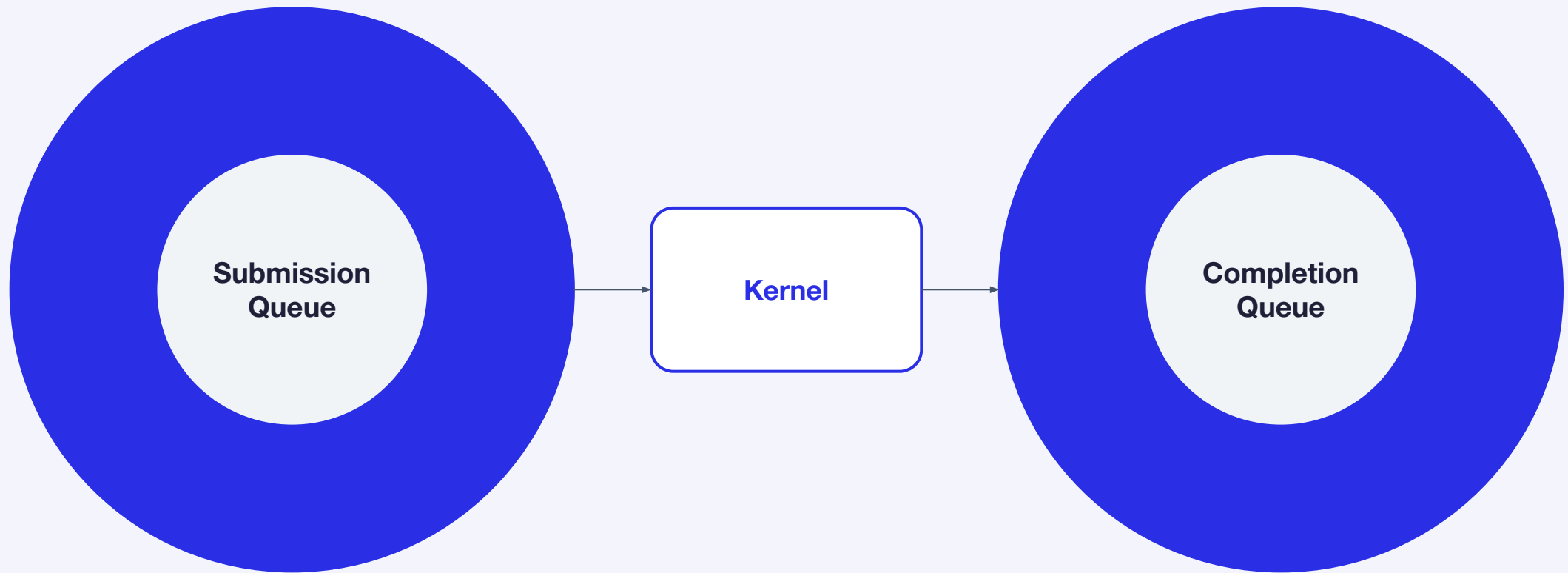
Towards Async Everything Part 1

Senders as the Lowest Layer

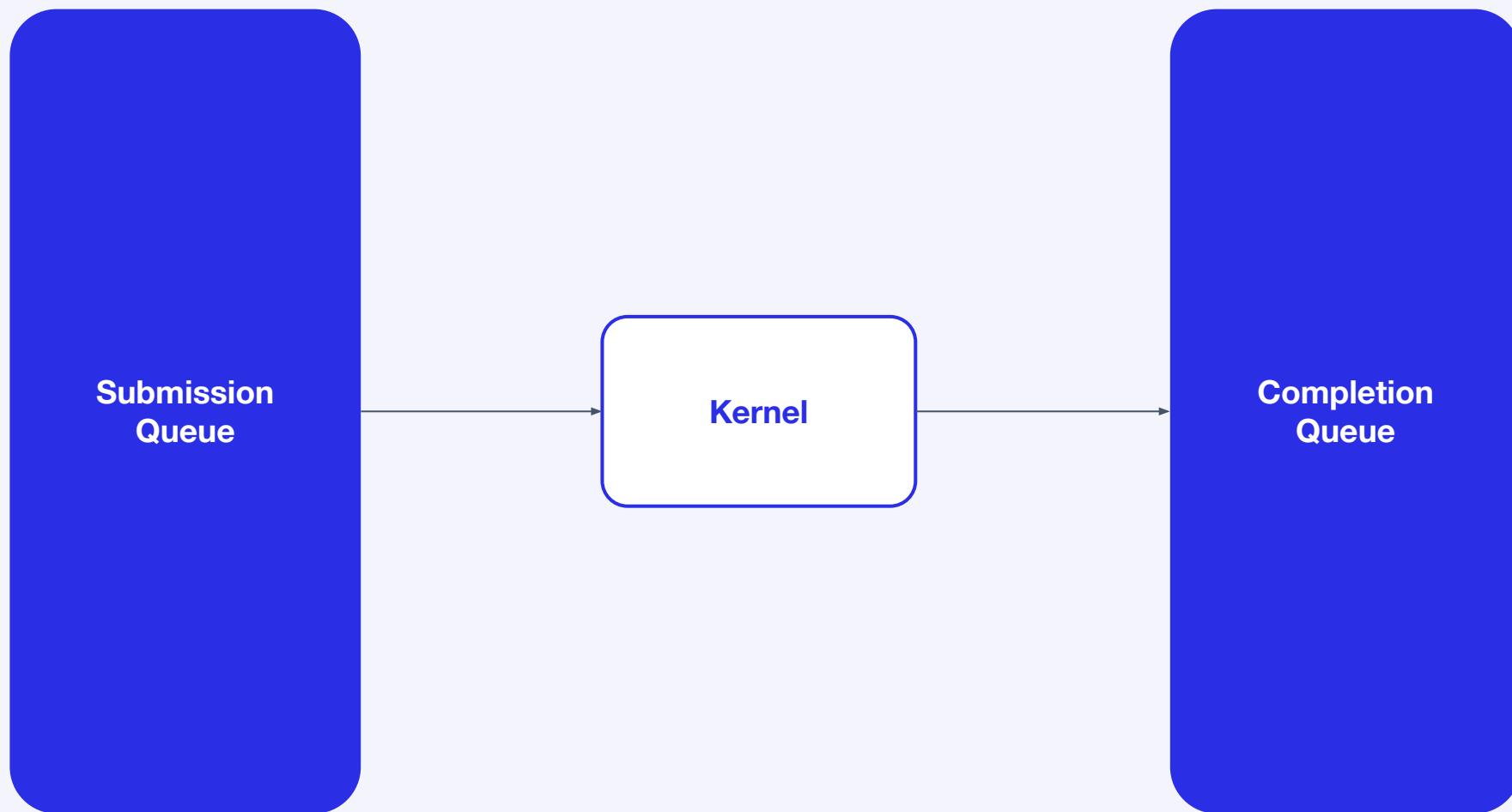
Robert Leahy



io_uring (Conceptually)



io_uring (Actually)



```
struct io_skring_offsets {  
    __u32 head;  
    __u32 tail;  
    __u32 ring_mask;  
    __u32 ring_entries;  
    __u32 flags;  
    __u32 dropped;  
    __u32 array;  
    __u32 resv1;  
    __u64 user_addr;  
};
```

```
struct io_cqring_offsets {  
    __u32 head;  
    __u32 tail;  
    __u32 ring_mask;  
    __u32 ring_entries;  
    __u32 overflow;  
    __u32 cqes;  
    __u32 flags;  
    __u32 resv1;  
    __u64 user_addr;  
};
```

```

template<typename T>
struct ring {
    std::atomic_ref<std::uint32_t> head;
    std::atomic_ref<std::uint32_t> tail;
    std::uint32_t ring_mask;
    T* array;

    template<typename Offsets>
    explicit ring(void* const ring, const Offsets& offsets, void* const array)
        noexcept
        : head(*reinterpret_cast<std::uint32_t*>(
            static_cast<std::byte*>(ring) + offsets.head)),
          tail(*reinterpret_cast<std::uint32_t*>(
            static_cast<std::byte*>(ring) + offsets.tail)),
          ring_mask(*reinterpret_cast<std::uint32_t*>(
            static_cast<std::byte*>(ring) + offsets.ring_mask)),
          array(static_cast<T*>(array))
    {}
};

```

```

template<typename T>
struct ring {
    std::atomic_ref<std::uint32_t> head;
    std::atomic_ref<std::uint32_t> tail;
    std::uint32_t          ring_mask;
    T*                    array;

    template<typename Offsets>
    explicit ring(void* const ring, const Offsets& offsets, void* const array)
        noexcept
        : head(*reinterpret_cast<std::uint32_t*>(
            static_cast<std::byte*>(ring) + offsets.head)),
          tail(*reinterpret_cast<std::uint32_t*>(
            static_cast<std::byte*>(ring) + offsets.tail)),
          ring_mask(*reinterpret_cast<std::uint32_t*>(
            static_cast<std::byte*>(ring) + offsets.ring_mask)),
          array(static_cast<T*>(array))
    {}
};

```

```
::io_uring_sqe* get_sqe() noexcept {  
    const auto tail = submission_queue_.tail.load(std::memory_order_relaxed);  
    const auto head = submission_queue_.head.load(std::memory_order_acquire);  
    const auto u     = tail & submission_queue_.ring_mask;  
    if (  
        (head != tail) &&  
        ((head & submission_queue_.ring_mask) == u))  
    {  
        return nullptr;  
    }  
    return submission_queue_.array + u;  
}
```

```
submission_queue_.tail.fetch_add(1, std::memory_order_release);
```

```
void complete_() noexcept {
    for (;;) {
        const auto head = completion_queue_.head.load(std::memory_order_relaxed);
        const auto tail = completion_queue_.tail.load(std::memory_order_acquire);
        if (head == tail) {
            return;
        }
        const auto cqe =
            completion_queue_.array + (head & completion_queue_.ring_mask);
        // ...
        completion_queue_.head.store(head + 1, std::memory_order_release);
    }
}
```

```
std::atomic<schedule_base_type_*> scheduled_head_;  
std::atomic<schedule_base_type_*> scheduled_tail_;
```

```
const auto prev = scheduled_tail_.exchange(&base, std::memory_order_acq_rel);  
if (prev) {  
    prev->next.store(&base, std::memory_order_release);  
} else {  
    scheduled_head_.store(&base, std::memory_order_release);  
}
```

```
const auto prev = scheduled_tail_.exchange(&base, std::memory_order_acq_rel);
if (prev) {
    prev->next.store(&base, std::memory_order_release);
} else {
    scheduled_head_.store(&base, std::memory_order_release);
}
const std::uint64_t to_write = 1;
::write(scheduler_fd_.native_handle(), &to_write, sizeof(to_write));
```

NAME [top](#)

`eventfd` – create a file descriptor for event notification

LIBRARY [top](#)

Standard C library (*libc*, *-lc*)

SYNOPSIS [top](#)

```
#include <sys/eventfd.h>
```

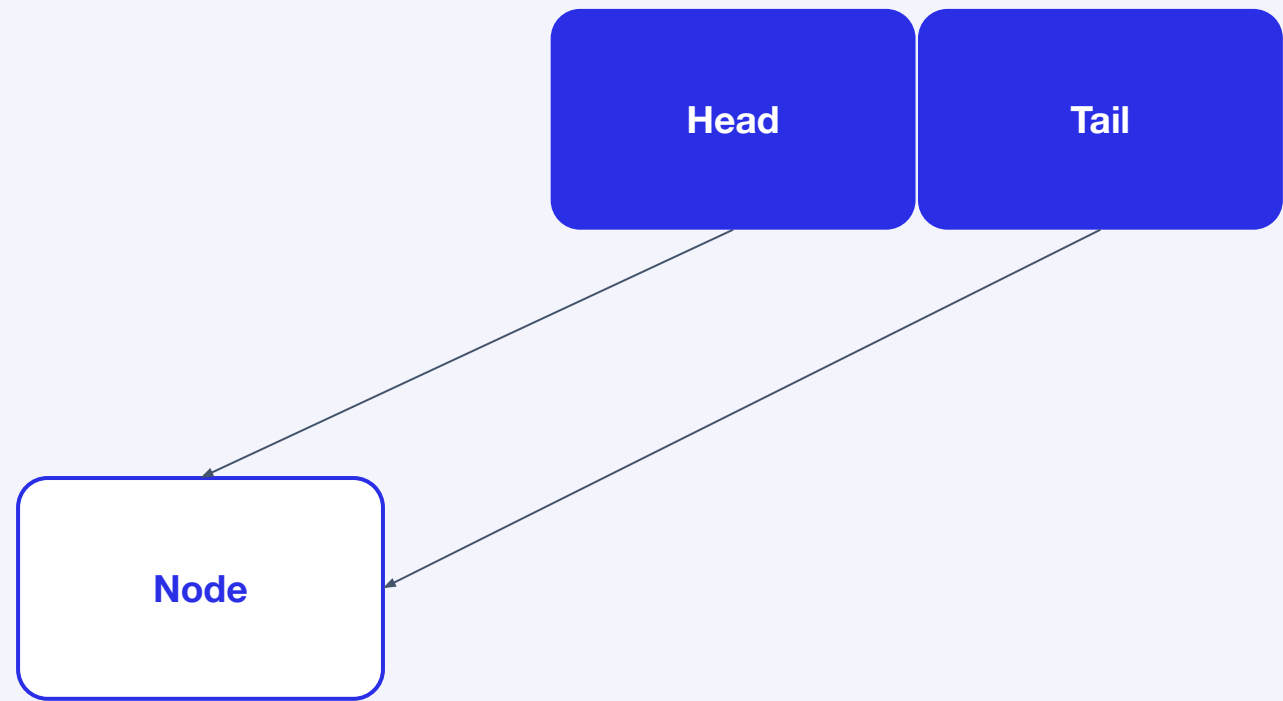
```
int eventfd(unsigned int initval, int flags);
```

DESCRIPTION [top](#)

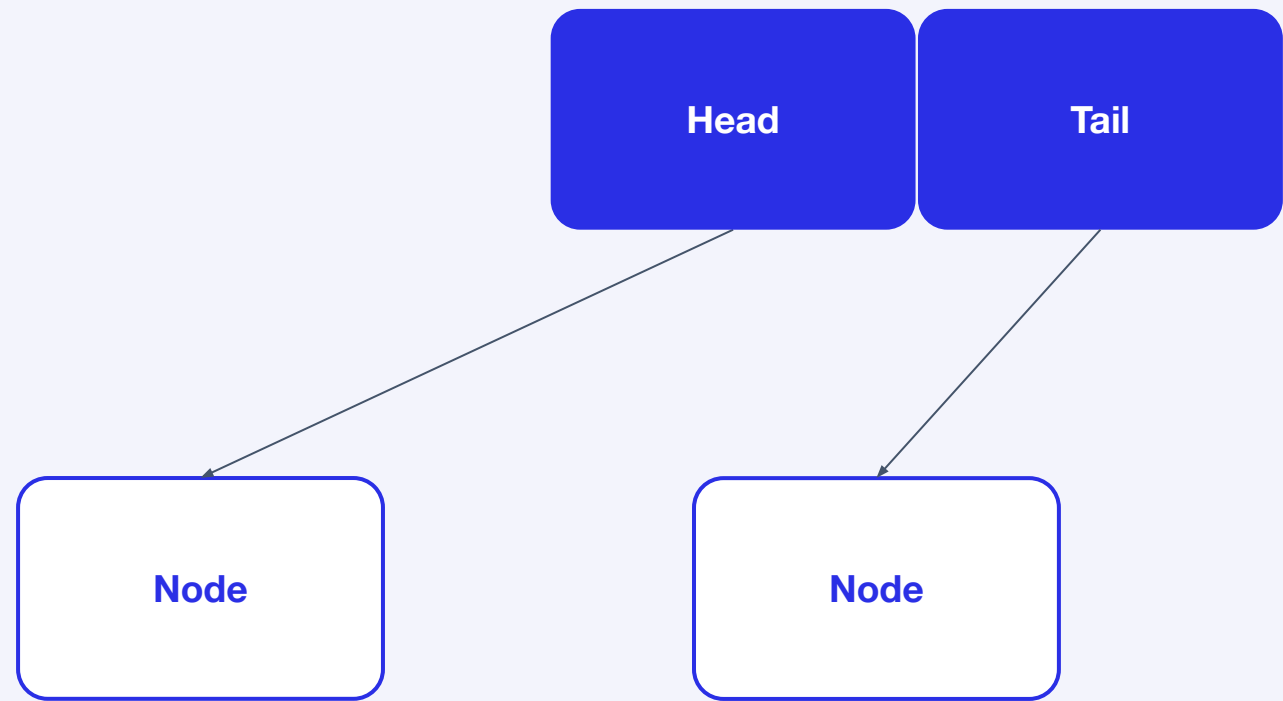
```
const auto prev = scheduled_tail_.exchange(&base, std::memory_order_acq_rel);
if (prev) {
    prev->next.store(&base, std::memory_order_release);
} else {
    scheduled_head_.store(&base, std::memory_order_release);
}
const std::uint64_t to_write = 1;
::write(scheduler_fd_.native_handle(), &to_write, sizeof(to_write));
```

```
const auto prev = scheduled_tail_.exchange(&base, std::memory_order_acq_rel);
if (prev) {
    prev->next.store(&base, std::memory_order_release);
} else {
    scheduled_head_.store(&base, std::memory_order_release);
}
const std::uint64_t to_write = 1;
::write(scheduler_fd_.native_handle(), &to_write, sizeof(to_write));
```

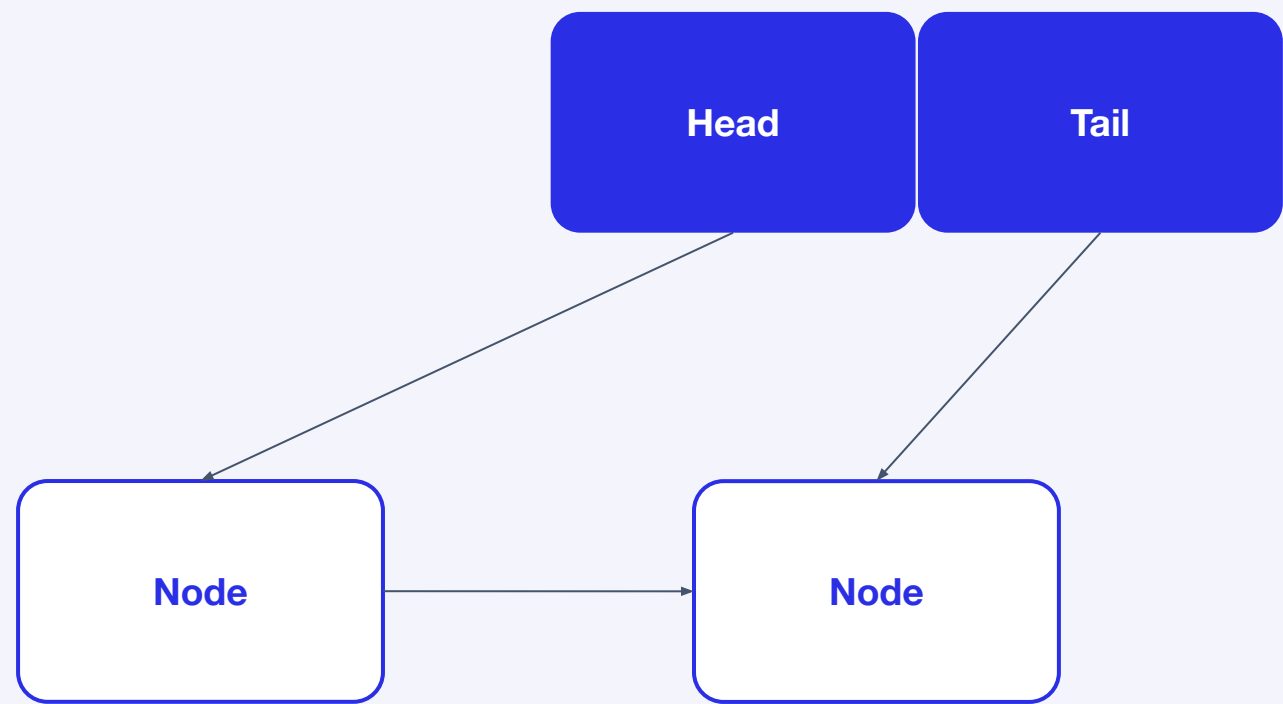
Momentary Inconsistency



Momentary Inconsistency



Momentary Inconsistency



```
void dispatch_(std::uint64_t count) noexcept {  
    auto curr = scheduled_head_.load(std::memory_order_acquire);  
    while (!curr) {  
        curr = scheduled_head_.load(std::memory_order_acquire);  
    }  
    // ...  
}
```

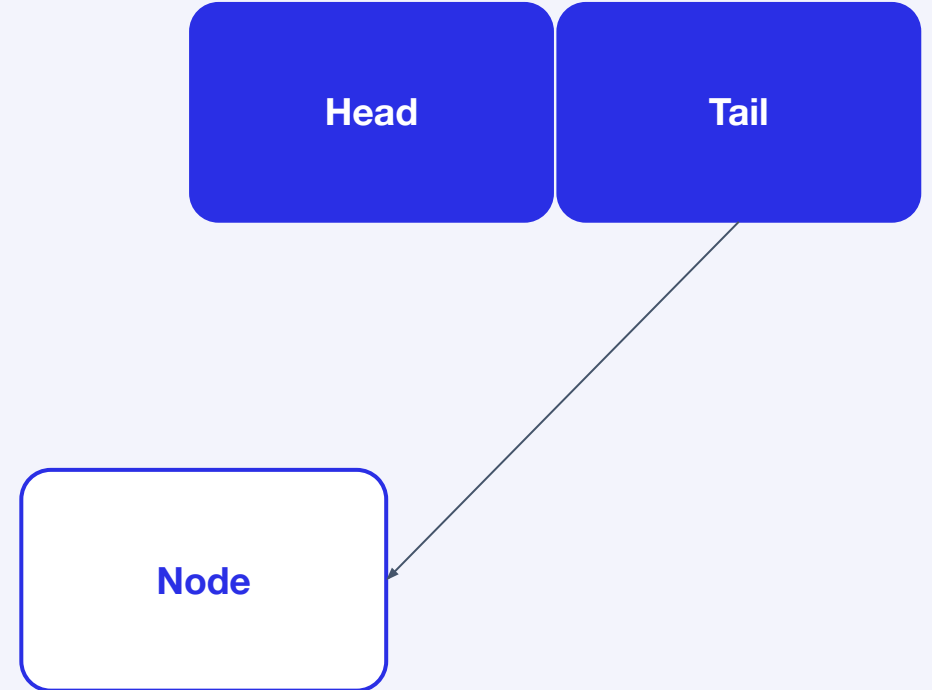
Why is Null Handling Necessary?

Head

Tail

Why is Null Handling Necessary?

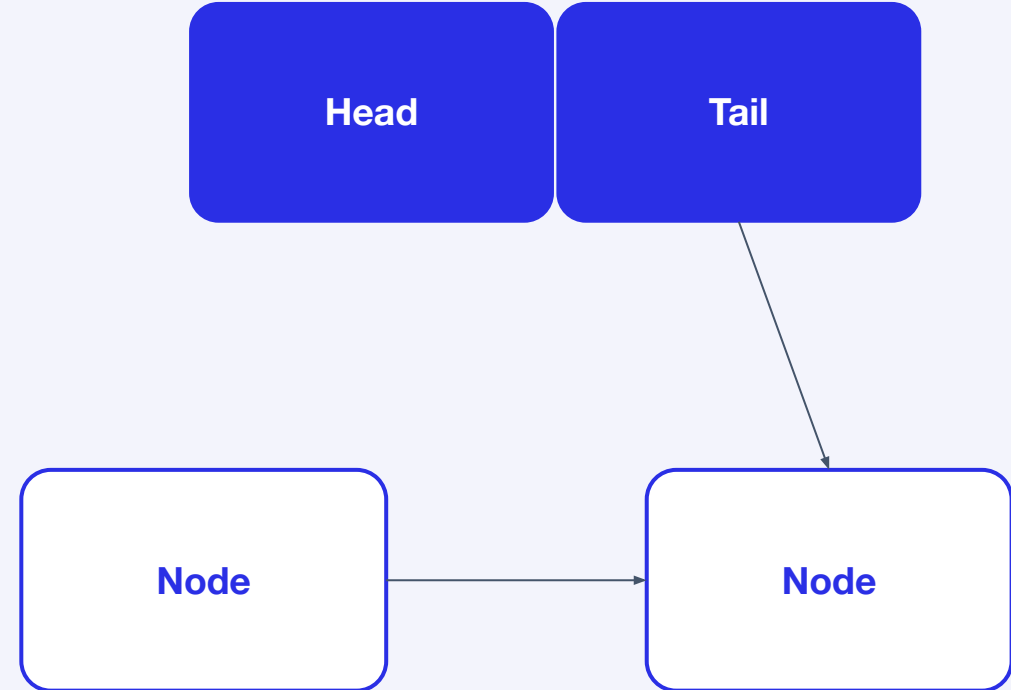
01 Thread A publishes its node to tail



Why is Null Handling Necessary?

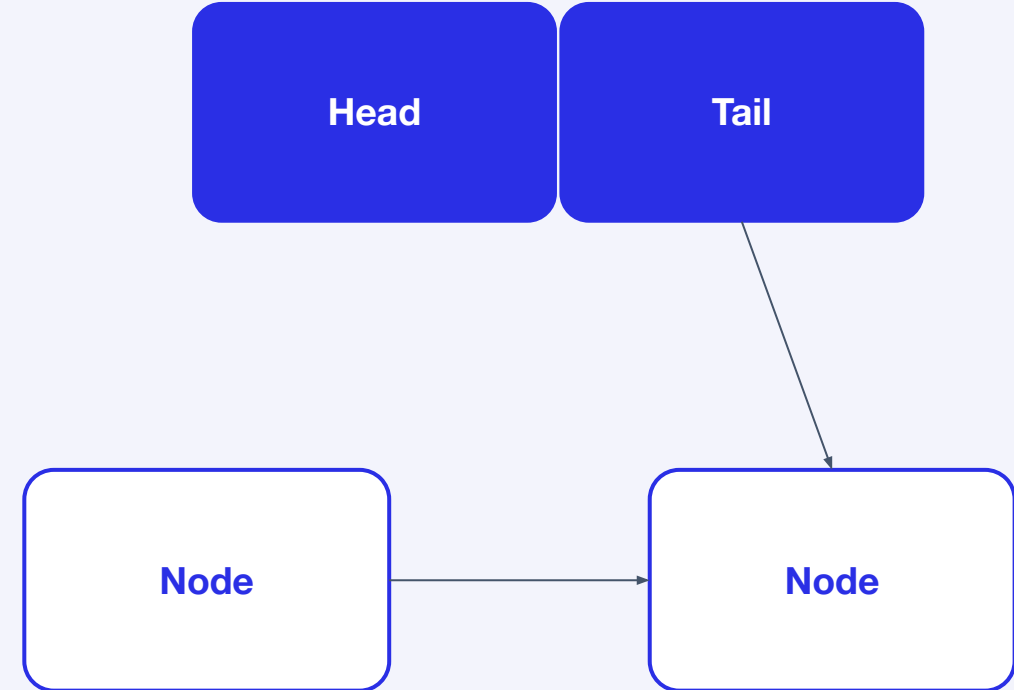
01 Thread A publishes its node to tail

02 Thread B publishes its node and writes to the eventfd



Why is Null Handling Necessary?

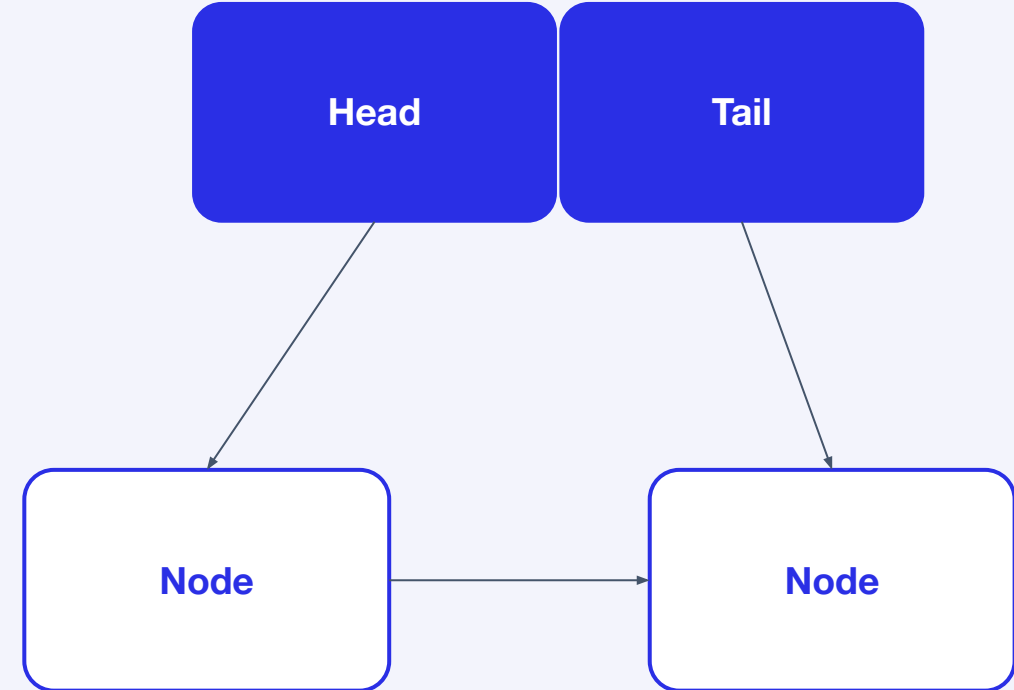
- 01** Thread A publishes its node to tail
- 02** Thread B publishes its node and writes to the eventfd
- 03** Thread C reads a count of one from the eventfd



```
void dispatch_(std::uint64_t count) noexcept {  
    auto curr = scheduled_head_.load(std::memory_order_acquire);  
    if (!curr) {  
        return;  
    }  
    // ...  
}
```

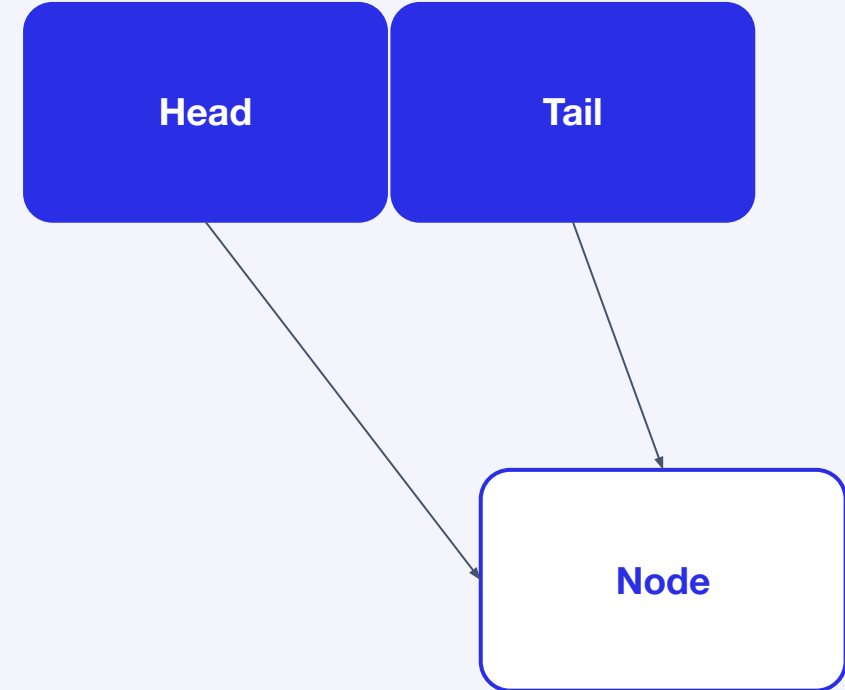
Why Not Return on Null?

- 01** Thread A publishes its node to tail
- 02** Thread B publishes its node and writes to the eventfd
- 03** Thread C reads a count of one from the eventfd
- 04** Thread A completes publishing and writes to eventfd



Why Not Return on Null?

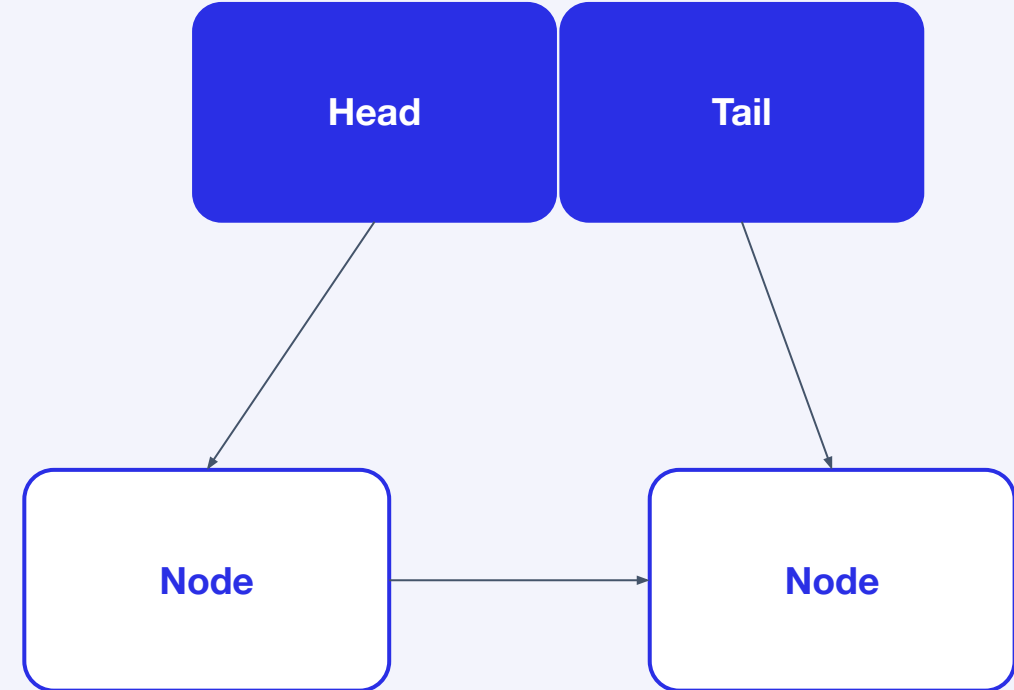
- 01** Thread A publishes its node to tail
- 02** Thread B publishes its node and writes to the eventfd
- 03** Thread C reads a count of one from the eventfd
- 04** Thread A completes publishing and writes to eventfd
- 05** Thread C reads a count of one from the eventfd



Why Not Consume Eagerly?

01 Thread A publishes its node and writes to the eventfd

02 Thread B publishes its node

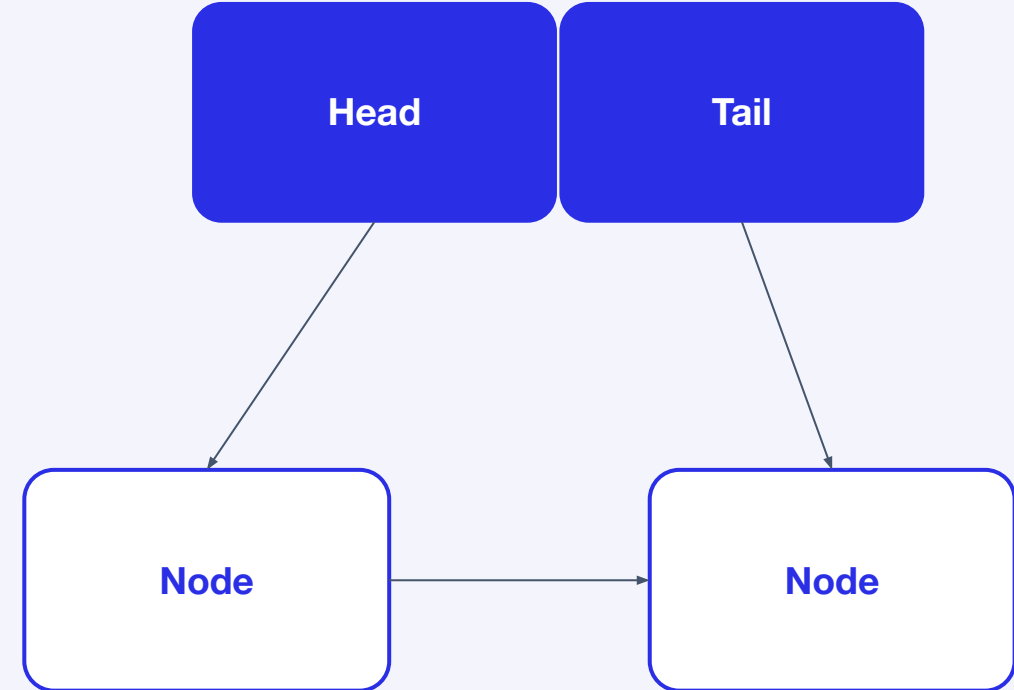


Why Not Consume Eagerly?

01 Thread A publishes its node and writes to the eventfd

02 Thread B publishes its node

03 Thread C reads a count of one from the eventfd



Why Not Consume Eagerly?

01 Thread A publishes its node and writes to the eventfd

02 Thread B publishes its node

03 Thread C reads a count of one from the eventfd

Head

Tail

Why Not Consume Eagerly?

01 Thread A publishes its node and writes to the eventfd

02 Thread B publishes its node

03 Thread C reads a count of one from the eventfd

04 Work is done, context is destroyed

Why Not Consume Eagerly?

01 Thread A publishes its node and writes to the eventfd

02 Thread B publishes its node

03 Thread C reads a count of one from the eventfd

04 Work is done, context is destroyed

05 Thread B writes to the eventfd

Why Not Consume Eagerly?

01 Thread A publishes its node and writes to the eventfd

02 Thread B publishes its node

03 Thread C reads a count of one from the eventfd

04 Work is done, context is destroyed

05 Thread B writes to the eventfd, which no longer exists

Why Not Consume Eagerly?

- 01 Thread A publishes its node and writes to the eventfd
- 02 Thread B publishes its node
- 03 Thread C reads a count of one from the eventfd
- 04 Work is done, context is destroyed
- 05 Thread B writes to the eventfd, which no longer exists, **which is UB**

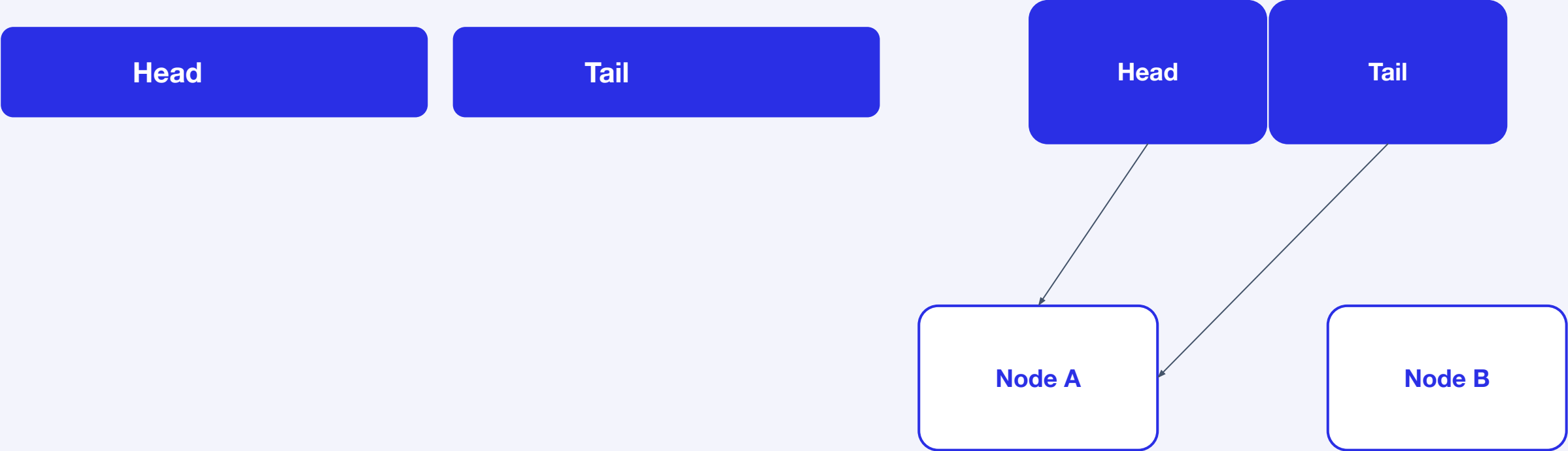
```
void dispatch_(std::uint64_t count) noexcept {  
    // ...  
    while (count > 1) {  
        for (;;) {  
            const auto next = curr->next.load(std::memory_order_acquire);  
            if (next) {  
                curr->complete(std::execution::set_value);  
                curr = next;  
                --count;  
                break;  
            }  
        }  
    }  
    // ...  
}
```

```
void dispatch_(std::uint64_t count) noexcept {  
    // ...  
    scheduled_head_.store(nullptr, std::memory_order_relaxed);  
    {  
        auto expected = curr;  
        if (scheduled_tail_.compare_exchange_strong(  
            expected,  
            nullptr,  
            std::memory_order_release))  
        {  
            curr->complete(std::execution::set_value);  
            return;  
        }  
    }  
    // ...  
}
```

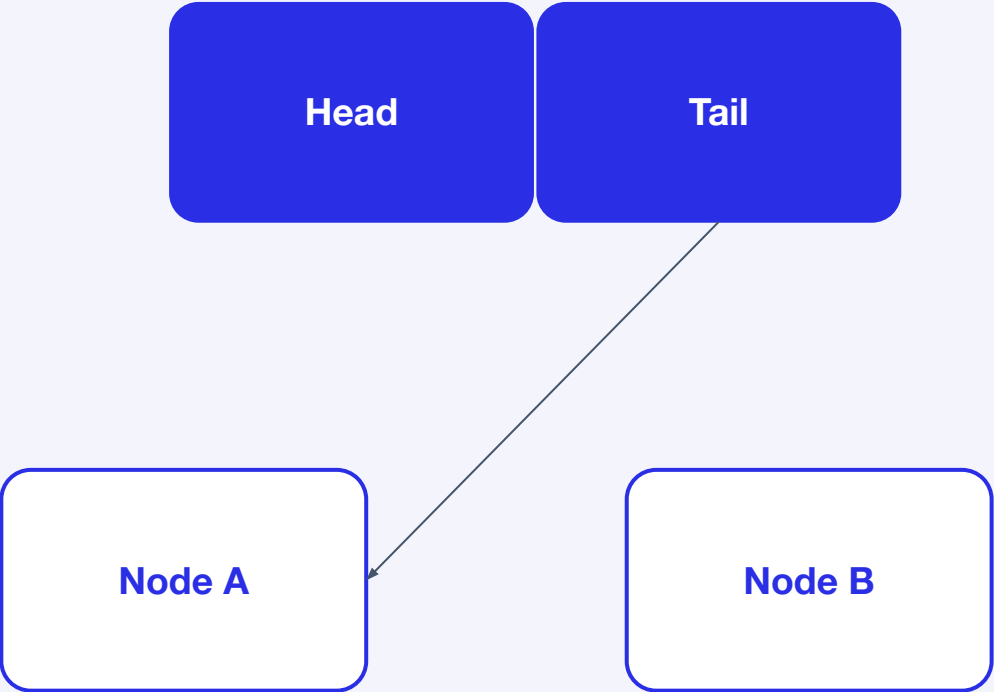
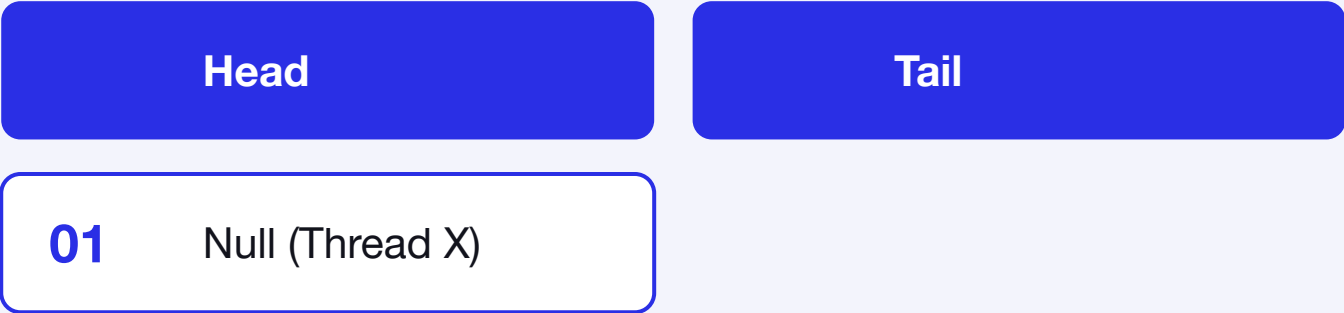
```
void dispatch_(std::uint64_t count) noexcept {  
    // ...  
    scheduled_head_.store(nullptr, std::memory_order_relaxed);  
    {  
        auto expected = curr;  
        if (scheduled_tail_.compare_exchange_strong(  
            expected,  
            nullptr,  
            std::memory_order_release))  
        {  
            curr->complete(std::execution::set_value);  
            return;  
        }  
    }  
    // ...  
}
```

```
void dispatch_(std::uint64_t count) noexcept {  
    // ...  
    scheduled_head_.store(nullptr, std::memory_order_relaxed);  
    {  
        auto expected = curr;  
        if (scheduled_tail_.compare_exchange_strong(  
            expected,  
            nullptr,  
            std::memory_order_release))  
        {  
            curr->complete(std::execution::set_value);  
            return;  
        }  
    }  
    // ...  
}
```

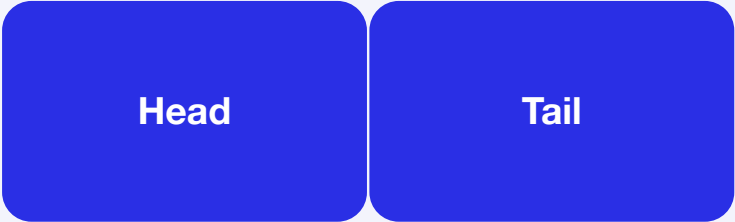
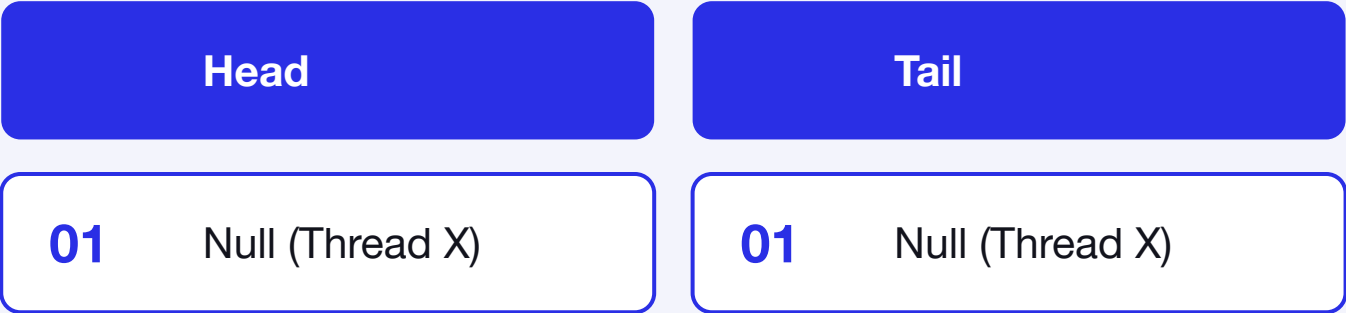
Eliminating Incorrect Modification Orders



Eliminating Incorrect Modification Orders



Eliminating Incorrect Modification Orders

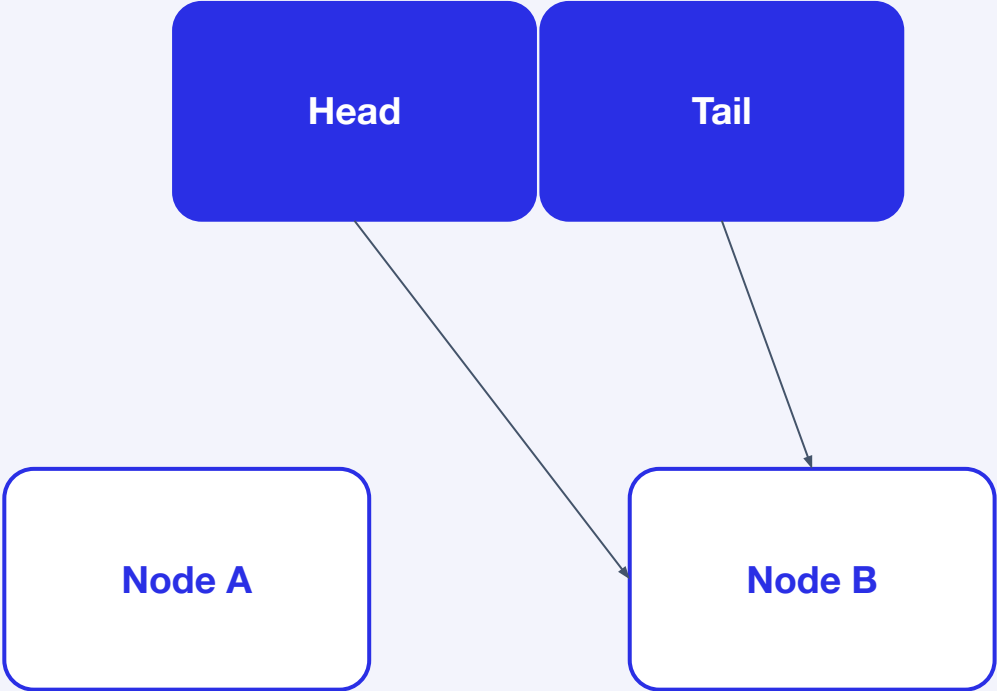


Eliminating Incorrect Modification Orders

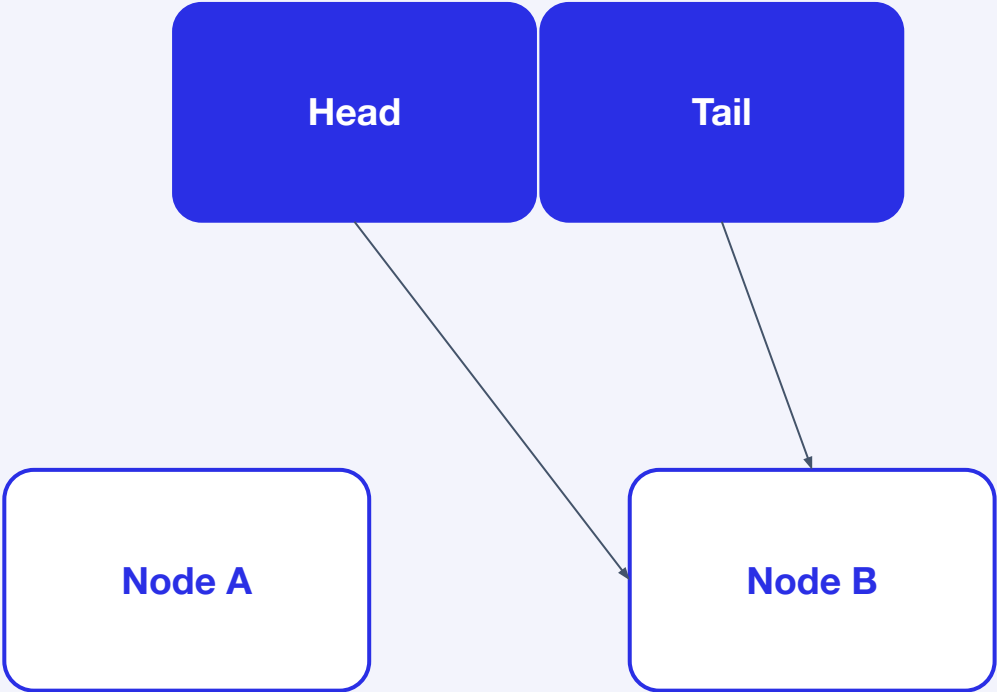
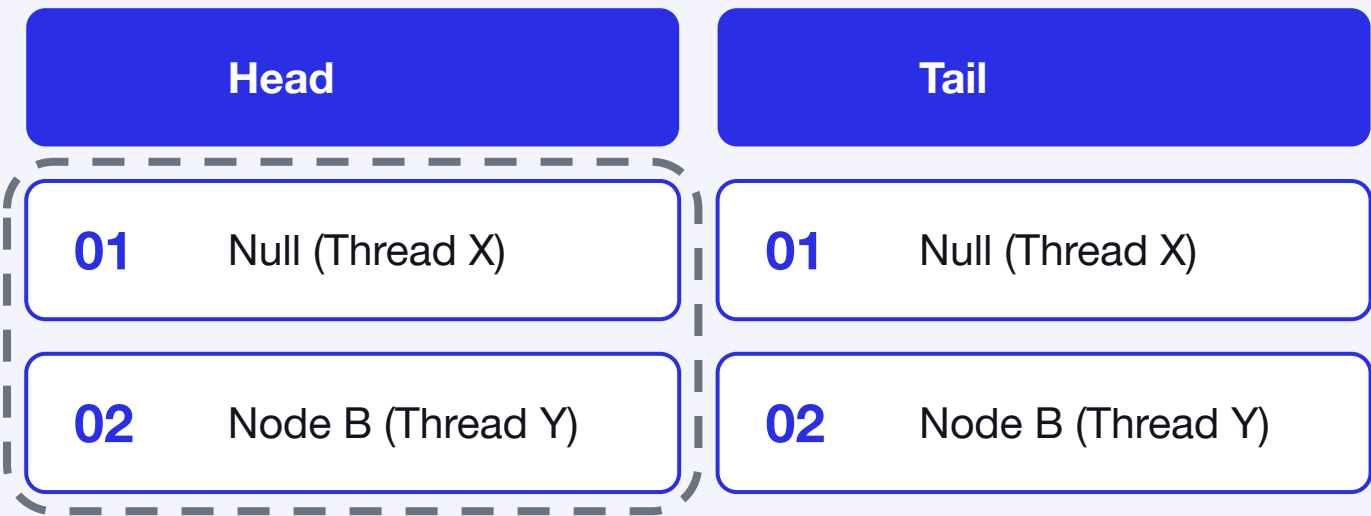


Eliminating Incorrect Modification Orders

Head	Tail
01 Null (Thread X)	01 Null (Thread X)
02 Node B (Thread Y)	02 Node B (Thread Y)



Eliminating Incorrect Modification Orders



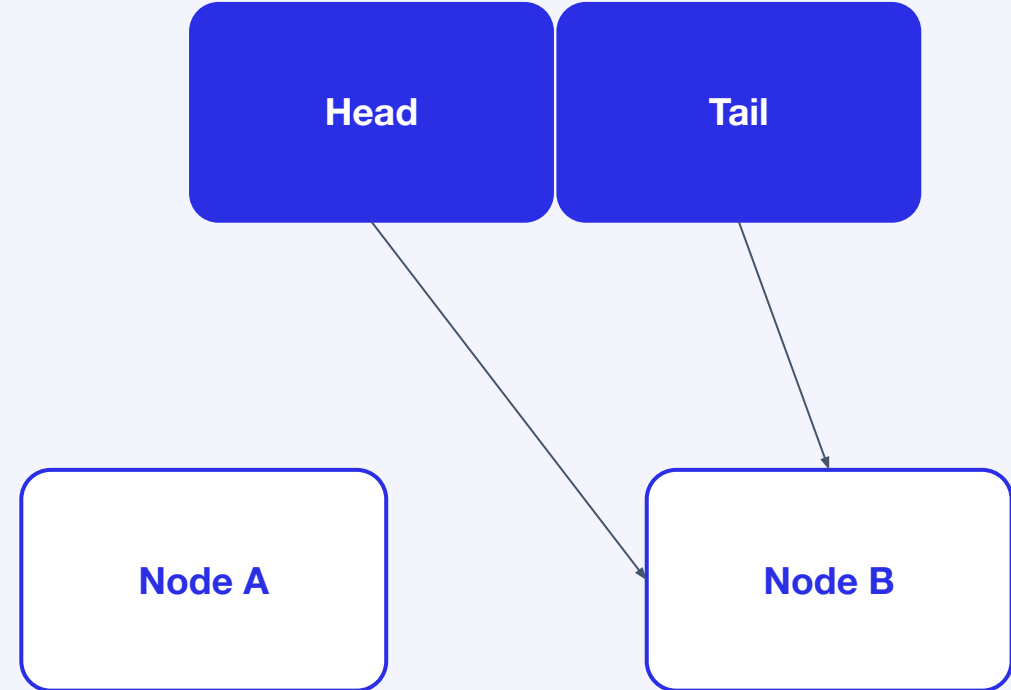
Total Order of Modifications

01 Thread X: Head $\rightarrow$ Null

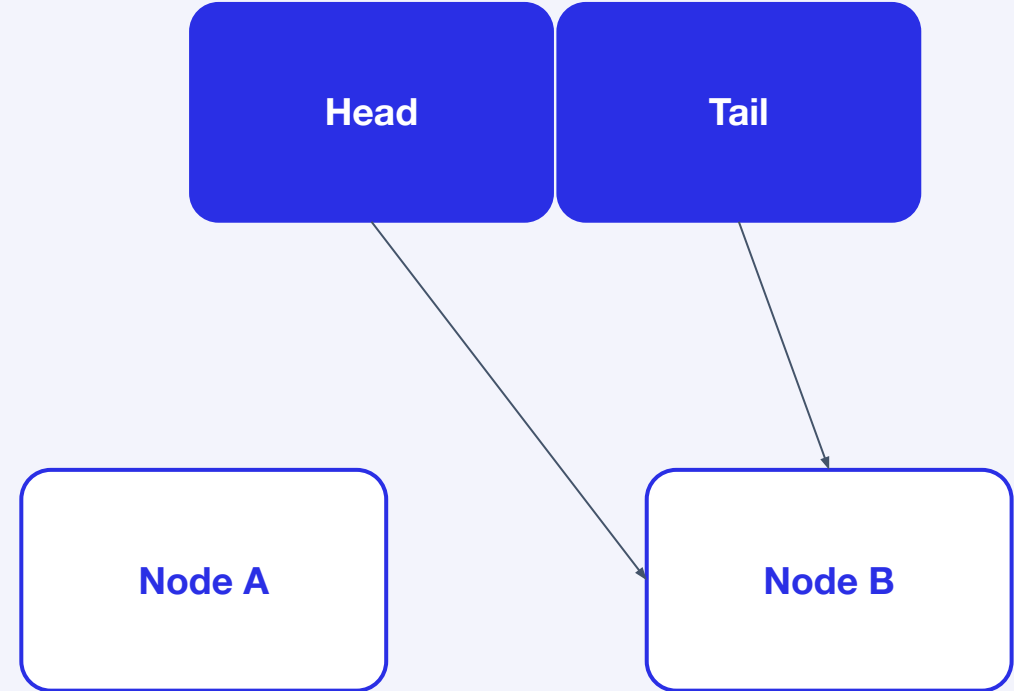
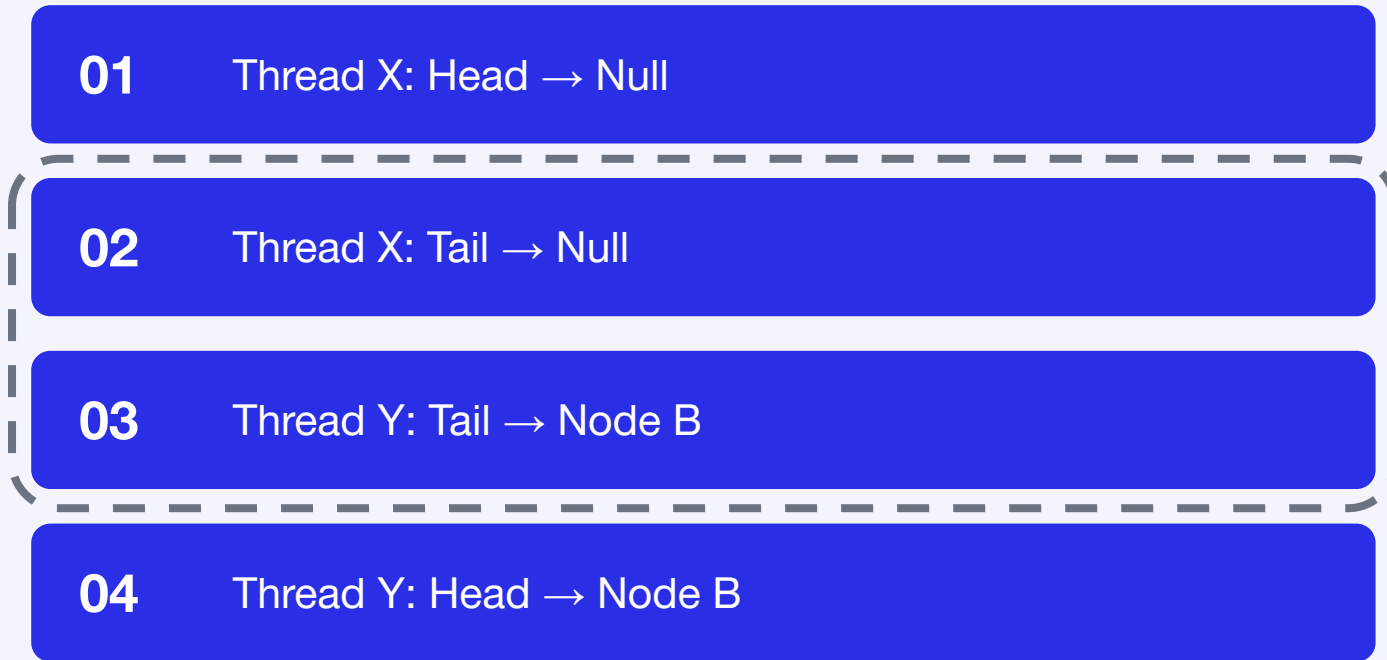
02 Thread X: Tail $\rightarrow$ Null

03 Thread Y: Tail $\rightarrow$ Node B

04 Thread Y: Head $\rightarrow$ Node B



Total Order of Modifications



All modifications to a **particular atomic object** M occur in some particular total order, called the modification order of M .

§6.10.2.2 [intro.races]

```
void dispatch_(std::uint64_t count) noexcept {  
    // ...  
    scheduled_head_.store(nullptr, std::memory_order_relaxed);  
    {  
        auto expected = curr;  
        if (scheduled_tail_.compare_exchange_strong(  
            expected,  
            nullptr,  
            std::memory_order_release))  
        {  
            curr->complete(std::execution::set_value);  
            return;  
        }  
    }  
    // ...  
}
```

```
void dispatch_(std::uint64_t count) noexcept {  
    // ...  
    scheduled_head_.store(nullptr, std::memory_order_relaxed);  
    {  
        auto expected = curr;  
        if (scheduled_tail_.compare_exchange_strong(  
            expected,  
            nullptr,  
            std::memory_order_relaxed))  
        {  
            curr->complete(std::execution::set_value);  
            return;  
        }  
    }  
    // ...  
}
```

```
void dispatch_(std::uint64_t count) noexcept {  
    // ...  
    scheduled_head_.store(nullptr, std::memory_order_relaxed);  
    {  
        auto expected = curr;  
        if (scheduled_tail_.compare_exchange_strong(  
            expected,  
            nullptr,  
            std::memory_order_relaxed))  
        {  
            curr->complete(std::execution::set_value);  
            return;  
        }  
    }  
    // ...  
}
```

```
void dispatch_(std::uint64_t count) noexcept {  
    // ...  
  
    {  
        auto expected = curr;  
        if (scheduled_tail_.compare_exchange_strong(  
            expected,  
            nullptr,  
            std::memory_order_relaxed))  
        {  
            scheduled_head_.store(nullptr, std::memory_order_relaxed);  
            curr->complete(std::execution::set_value);  
            return;  
        }  
    }  
    scheduled_head_.store(nullptr, std::memory_order_relaxed);  
    // ...  
}
```

If an operation A that modifies an atomic object M happens before an operation B that modifies M , then A is earlier than B in the modification order of M .

§6.10.2.2 [intro.races]

```
void dispatch_(std::uint64_t count) noexcept {  
    // ...  
    for (;;) {  
        const auto next = curr->next.load(std::memory_order_acquire);  
        if (next) {  
            scheduled_head_.store(next, std::memory_order_relaxed);  
            curr->complete(std::execution::set_value);  
            return;  
        }  
    }  
}
```

Towards Async Everything Part 1

Senders as the Lowest Layer

Robert Leahy

Towards Async Everything Part 2

Scopes, Construction, and Destruction

Robert Leahy



The Async Pattern Graveyard

Robert Leahy

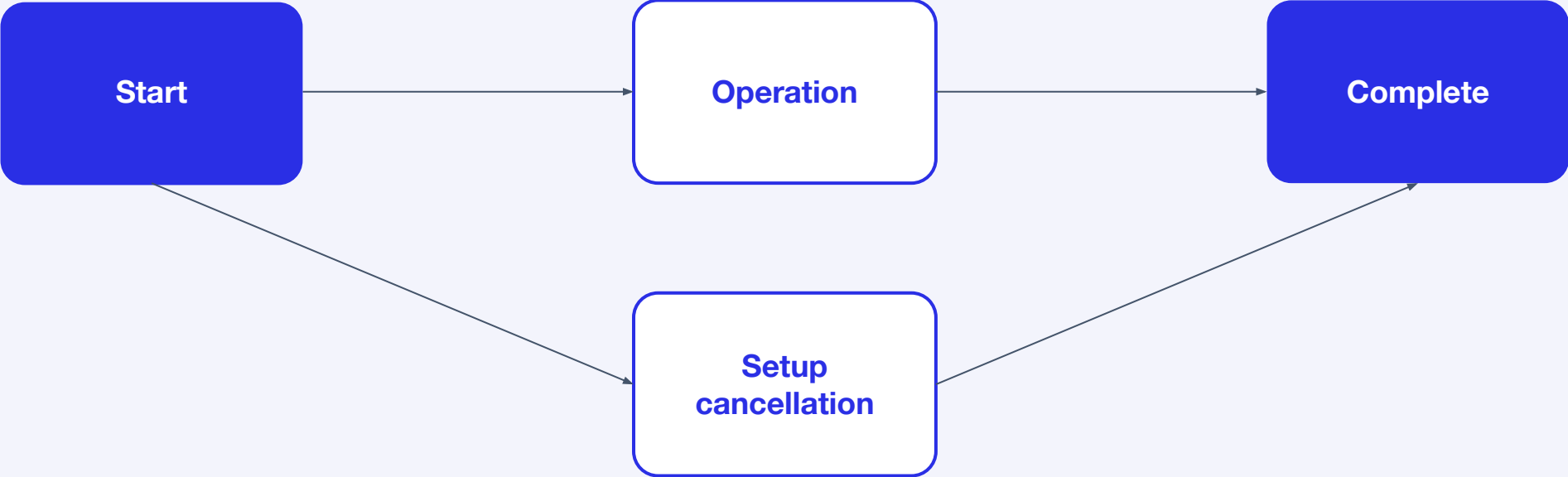


```
template<
    typename Signatures,
    typename Base = externally_completed_base<Signatures>,
    typename Init,
    typename Cancel,
    typename Classifier>
std::execution::sender auto externally_completed(
    Init init,
    Cancel cancel,
    Classifier classifier);
```

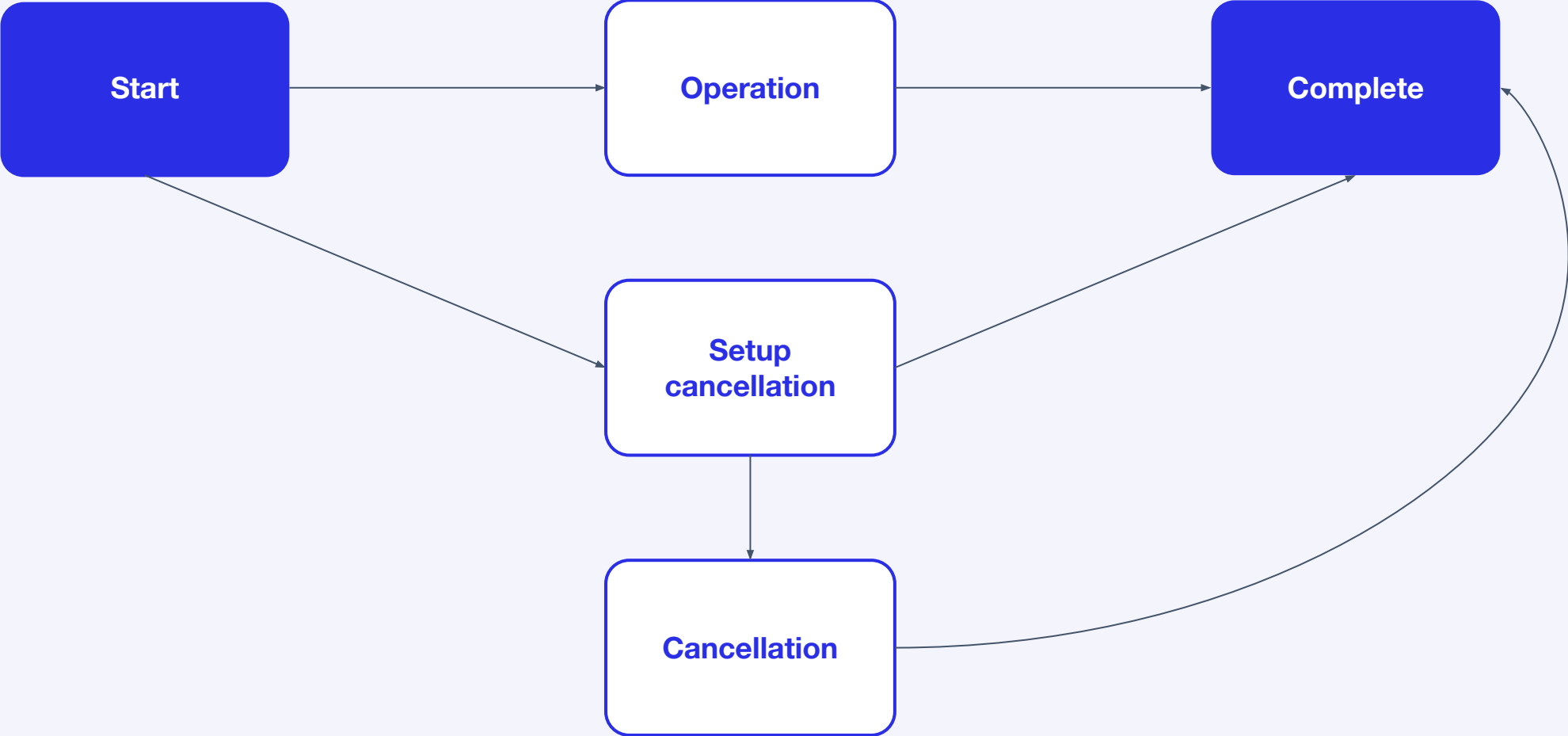
externally_completed



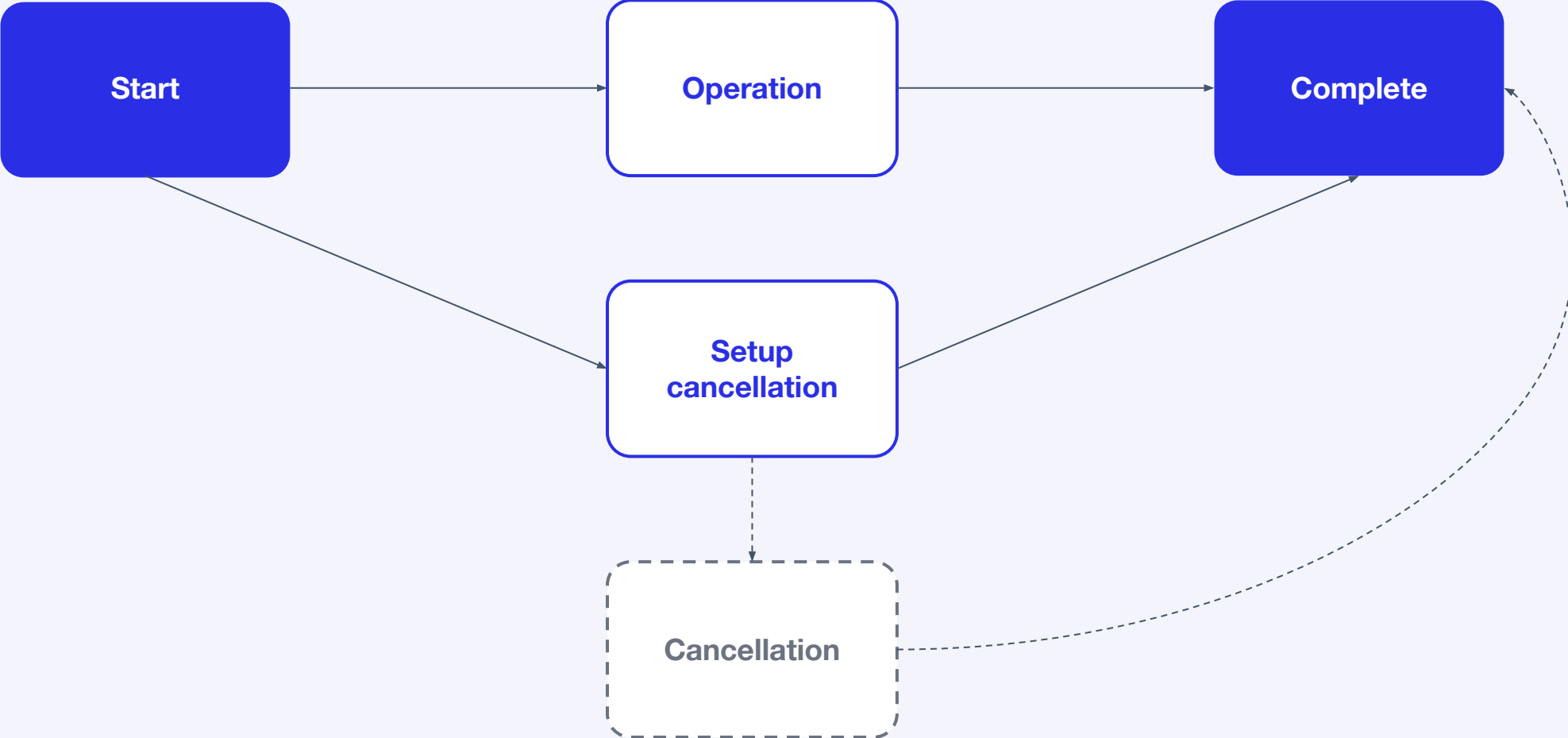
externally_completed



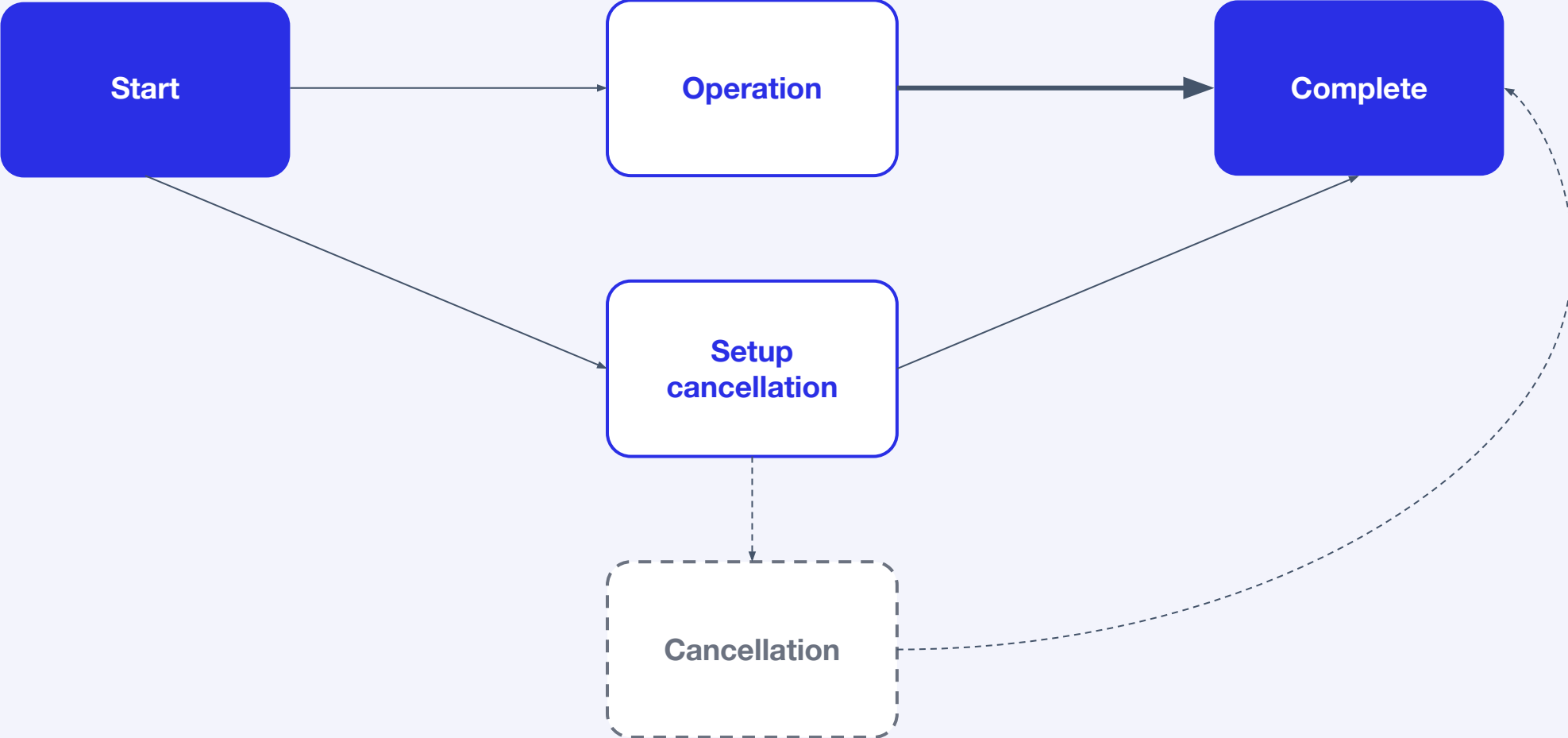
externally_completed



externally_completed



externally_completed



```
// One for start, one for the actual asynchronous operation  
std::atomic<unsigned> outstanding_{2};
```

```
constexpr void start() & noexcept {  
  
    try {  
        std::invoke(std::move(init_), static_cast<Base*>(*this));  
    } catch (...) {  
        if constexpr (!std::is_nothrow_invocable_v<Init, Base*>) {  
            std::execution::set_error(std::move(rcvr_), std::current_exception());  
            return;  
        }  
    }  
  
    callback_.emplace(  
        std::get_stop_token(std::execution::get_env(rcvr_)),  
        on_stop_request_type_{*this});  
    maybe_complete_();  
  
}
```

```
constexpr void stop_request_() noexcept {  
  
    if (outstanding_.fetch_add(1, std::memory_order_relaxed) == 0) {  
        return;  
    }  
  
    auto&& op = cancel_op_.emplace(  
        emplace-from([&]() noexcept {  
            return std::execution::connect(  
                std::invoke(std::move(cancel_), static_cast<Base&>(*this)),  
                cancel_receiver_type_{*this});  
            }));  
    std::execution::start(op);  
  
}
```

```
constexpr void maybe_complete_() noexcept {  
  
    if (outstanding_.fetch_sub(1, std::memory_order_release) == 1) {  
        std::atomic_thread_fence(std::memory_order_acquire);  
        callback_.reset();  
        std::move(storage_).complete(std::move(rcvr_));  
    }  
  
}
```

```
template<typename Tag, typename... Args>
constexpr void complete_(Tag t, Args&&... args) noexcept {

    if (outstanding_.load(std::memory_order_relaxed) == 1) {
        std::atomic_thread_fence(std::memory_order_acquire);
        callback_.reset();
        if (outstanding_.load(std::memory_order_acquire) == 1) {
            t(std::move(rcvr_), std::forward<Args>(args)...);
            return;
        }
    }

    storage_.arrive(t, std::forward<Args>(args)...);
    maybe_complete_();
}
```

```
enum class memory_order : unspecified {  
    relaxed,  
    acquire,  
    release,  
    acq_rel,  
    seq_cst  
};
```

```
enum class memory_order : unspecified {  
    relaxed,  
    acquire,  
    release,  
    acq_rel,  
    seq_cst  
};
```

Sequentially Consistent

Loads and Stores

- [...] `memory_order::seq_cst`: a store operation performs a release operation on the affected memory location.
- [...] `memory_order::seq_cst`: a load operation performs an acquire operation on the affected memory location.

§32.5.4 [atomics.order]

Total Order of Operations

There is a single total order S on all `memory_order::seq_cst` operations, including fences [...]

§32.5.4 [atomics.order]

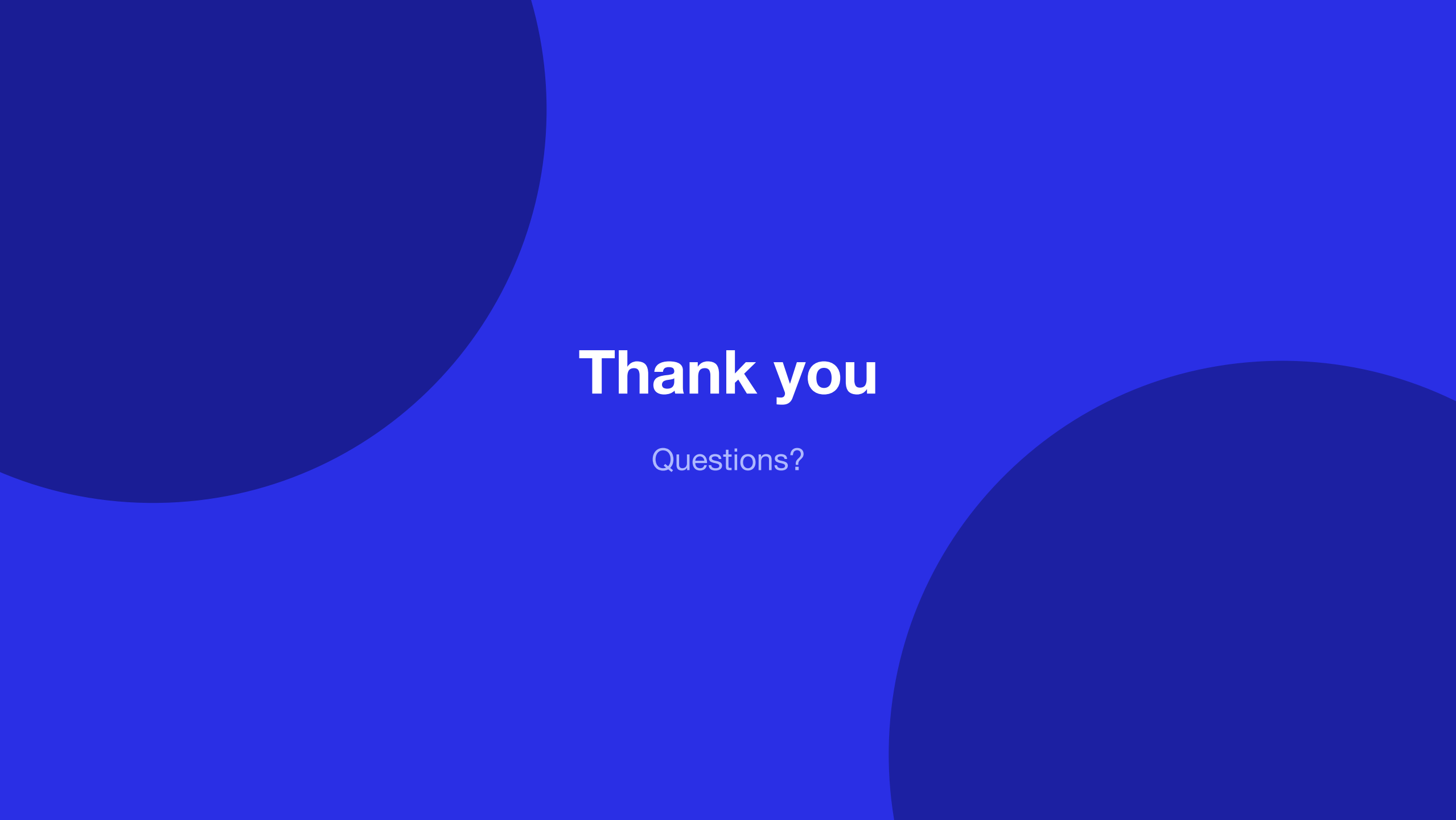


YAGNI

You Aren't Gonna Need It

Conclusion

- 01** Atomicity is different from synchronization.
- 02** Synchronization is provided by “happens before” relationships.
- 03** Locks let us reason about critical sections.
- 04** Lock-free programming forces us to reason about every pathological state.



Thank you

Questions?